

Code Agent 기본 & Engineering 발전과정 이해

CONTENTS

목차

CHAPTER 1

챕터의 목적

CHAPTER 2

CLAUDE.md

CHAPTER 3

Subagent

CHAPTER 4

Skill

CHAPTER 5

MCP

Chapter 01

챕터의 목적

하루만에 익히는 Code Agent

Claude Code의 기본 사용법을 익힌다.

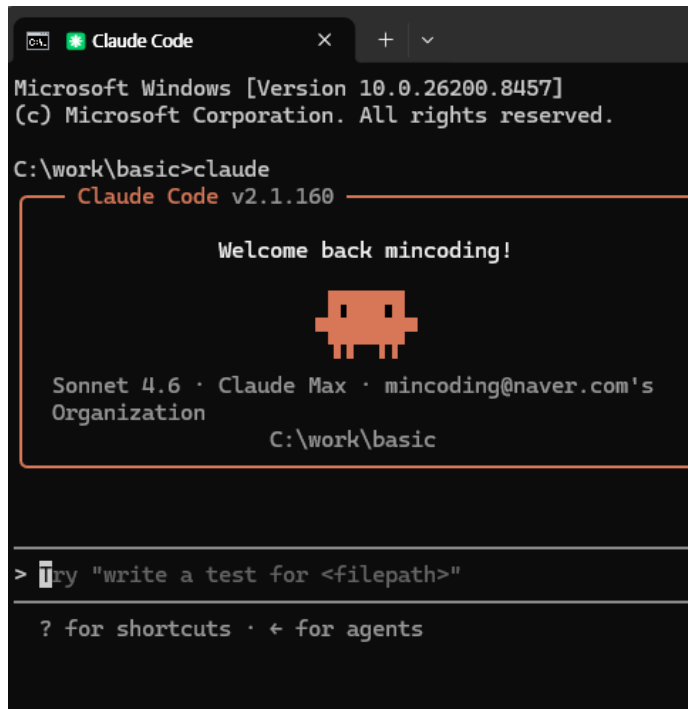
이 챕터에서는 프로그래밍 소스코드를 다루지 않고, 기본 사용법만 익혀본다.
Agentic Engineering 학습을 위한 최소한의 Claude Code 사용법을 리뷰한다.

커리큘럼

1. CLAUDE.md 사용법을 익히고,
2. 간단한 문서를 만들어본다.
3. 문서를 수정해보고, 작업을 취소하는 방법을 배운다.
4. Subagent를 사용해본다.
5. Skill와 MCP 개념을 익히고, 가볍게 다루어본다.

실습 환경

- 1) PowerShell + Claude Code로 CLI 기반으로 사용
- 2) vscode (Visual Studio Code)를 “문서에디터” 용도로만 사용
프로그래밍 용도가 아닌, 문서 Editor 용도로만 사용할 예정
설치 : <https://code.visualstudio.com/>



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

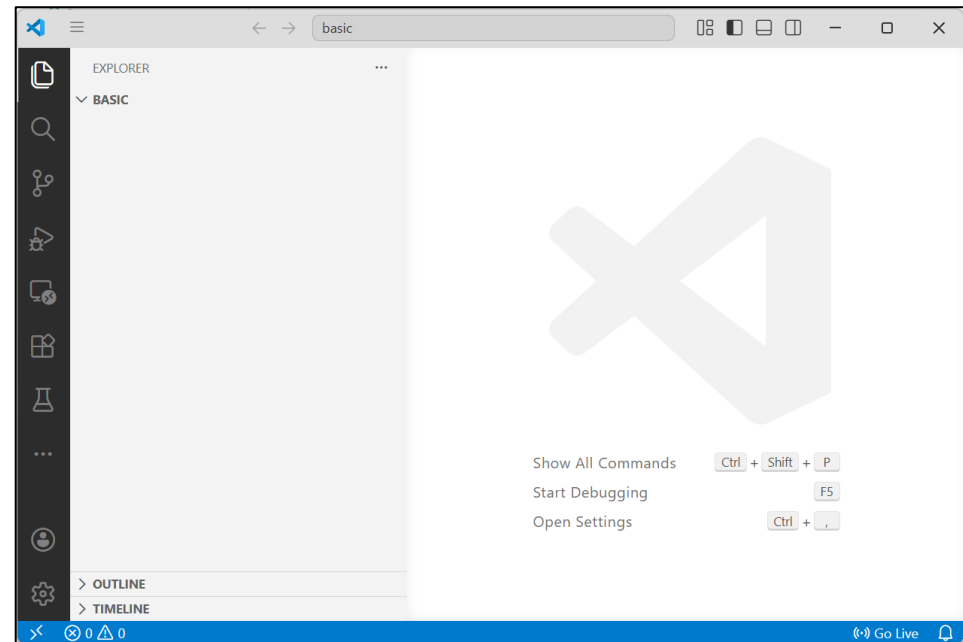
C:\work\basic>claude
Claude Code v2.1.160

Welcome back mincoding!

Sonnet 4.6 · Claude Max · mincoding@naver.com's Organization
C:\work\basic

> try "write a test for <filepath>"

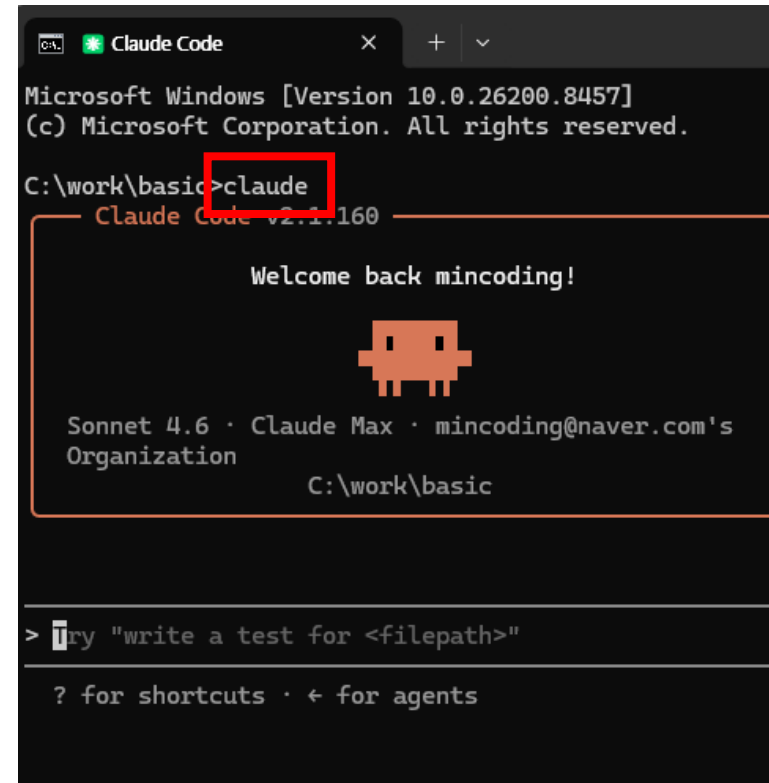
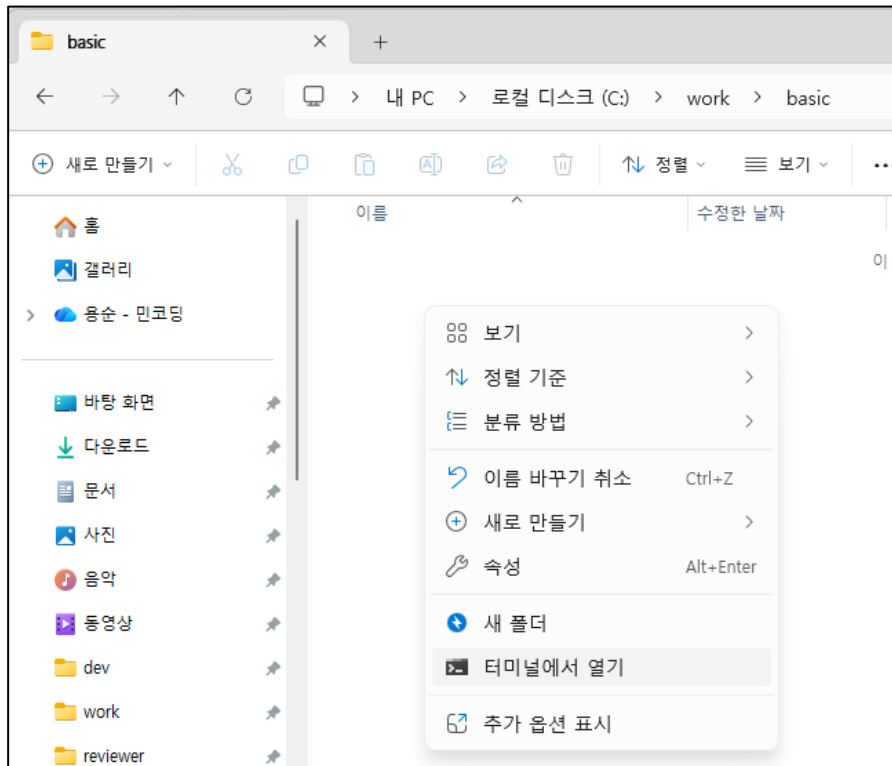
? for shortcuts · ← for agents
```



Claude 실행

프로젝트 폴더를 하나 생성 후, Claude를 실행한다.

c:\work\basic 폴더 생성 후, claude 실행



Chapter 02

CLAUDE.md

하루만에 익히는 Code Agent

- CLAUDE.md 파일이란?

Claude Code에서 사용되는 규칙 / 지침 파일

이곳에 내용을 기입하면, Claude는 사용자의 규칙(지침)으로 간주하고 행동한다.

- 사용처

Claude Code가 항상 참고해야할 내용을 CLAUDE.md에 적는다.

- 매번 프롬프트로 반복 설명할 필요가 없다.
- 이곳에 프로젝트에 대한 설명, 개발 방식, 팀의 규칙등을 정의한다.

생성해야 하는 위치

현재 열려있는 프로젝트의 Root 폴더에 둔다.

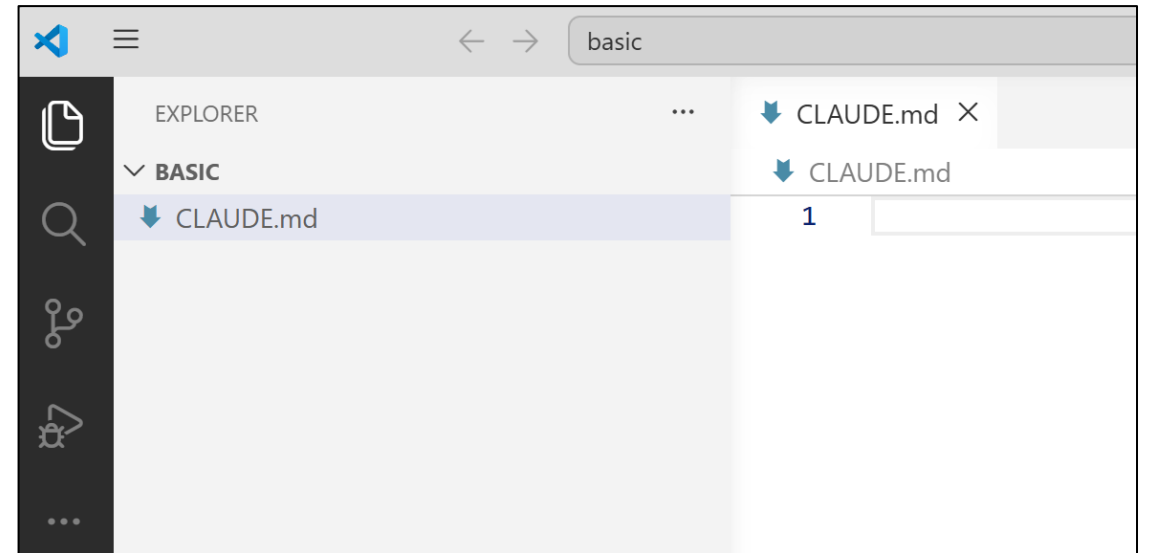
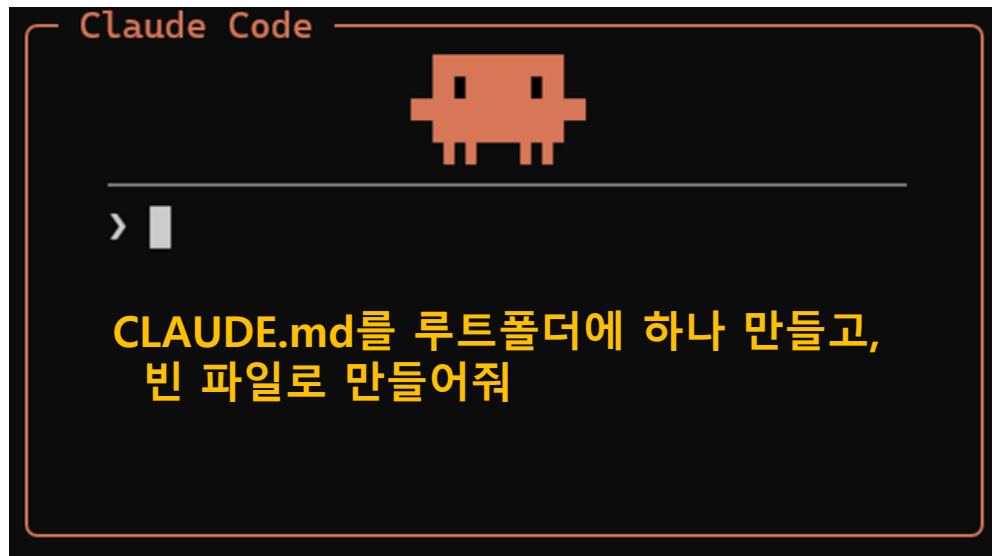
- 루트 폴더에 넣어두면, 프로젝트 전체에 규칙을 적용한다.
- 비유 : 프로젝트의 헌법

```
/my-project
├── CLAUDE.md
├── src/
├── test/
└── README.md
```

경로 구성 예시

CLAUDE.md 파일 생성

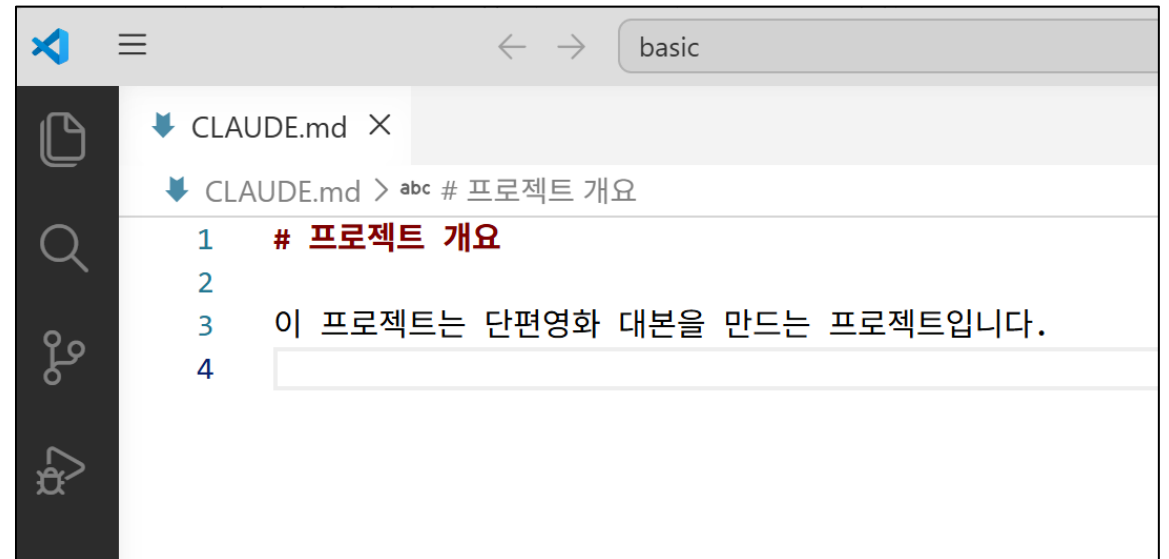
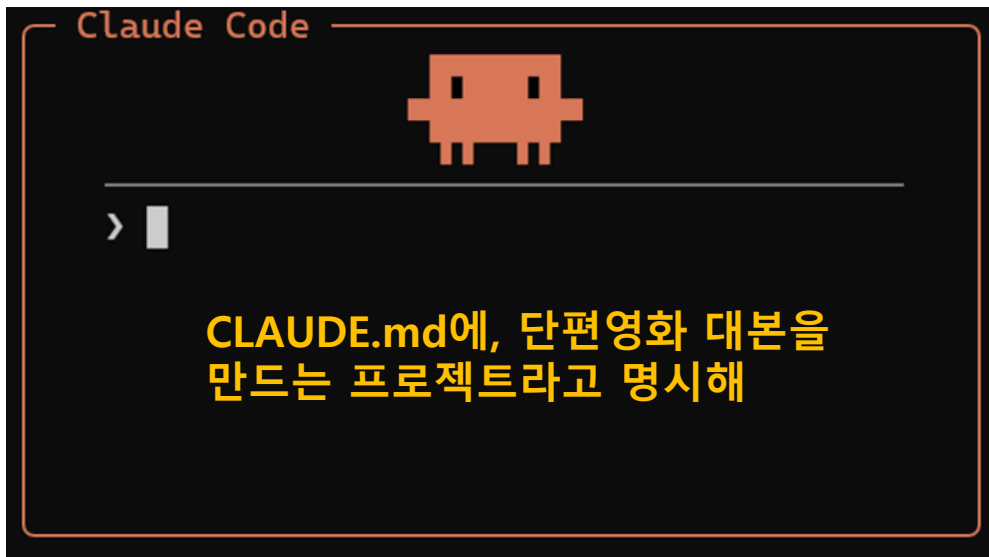
1. 클로드에게 지시한다.
 - CLAUDE.md 파일 생성



CLAUDE.md 기본 내용 작성

2. 클로드에게 지시한다.

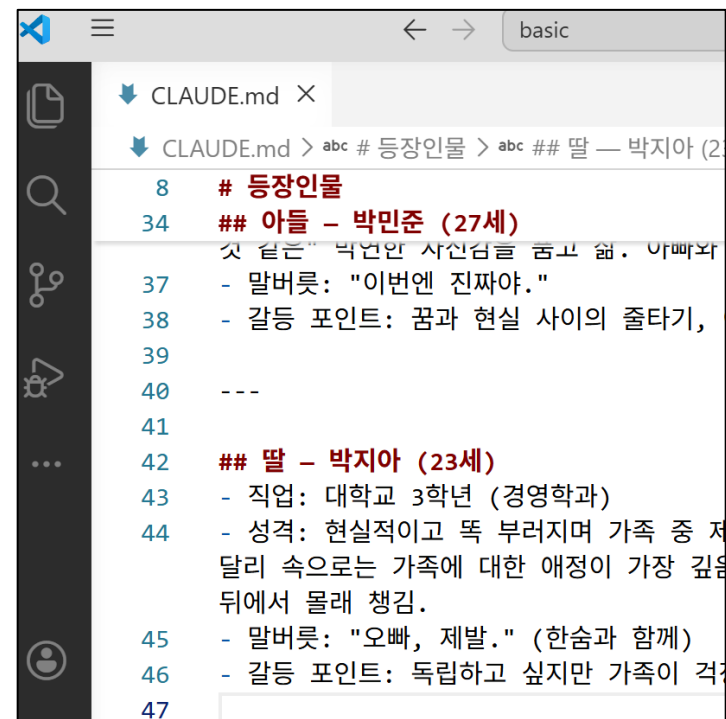
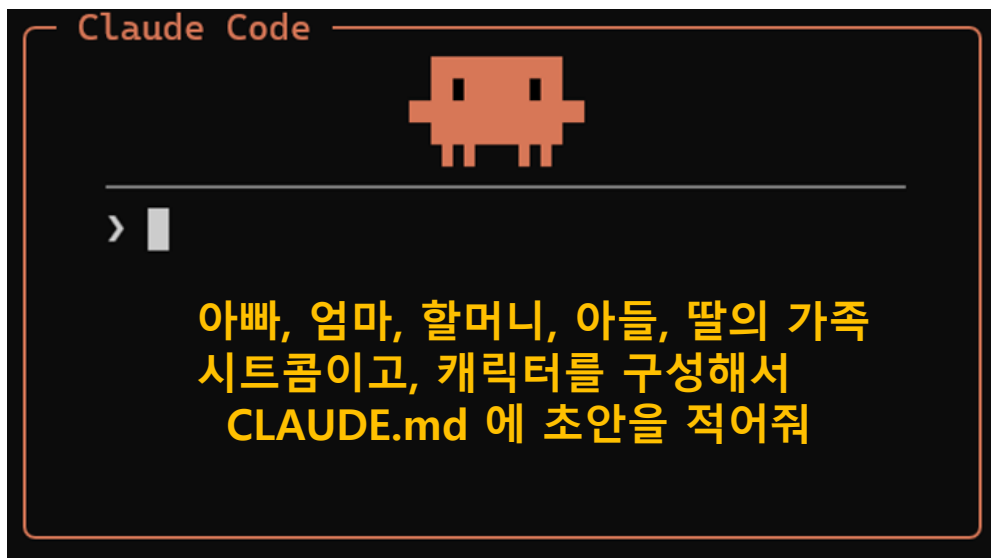
- CLAUDE.md에는 프로젝트의 목적을 적어준다.



CLAUDE.md 초안 작성 요청

3. 클로드에게 지시한다.

- CLAUDE.md에는 프로젝트 구성요소에 대한 설명을 적는다.
- 프로젝트가 지속적으로 발전하더라도, 변경이 없는 내용을 적는 것이 좋다.

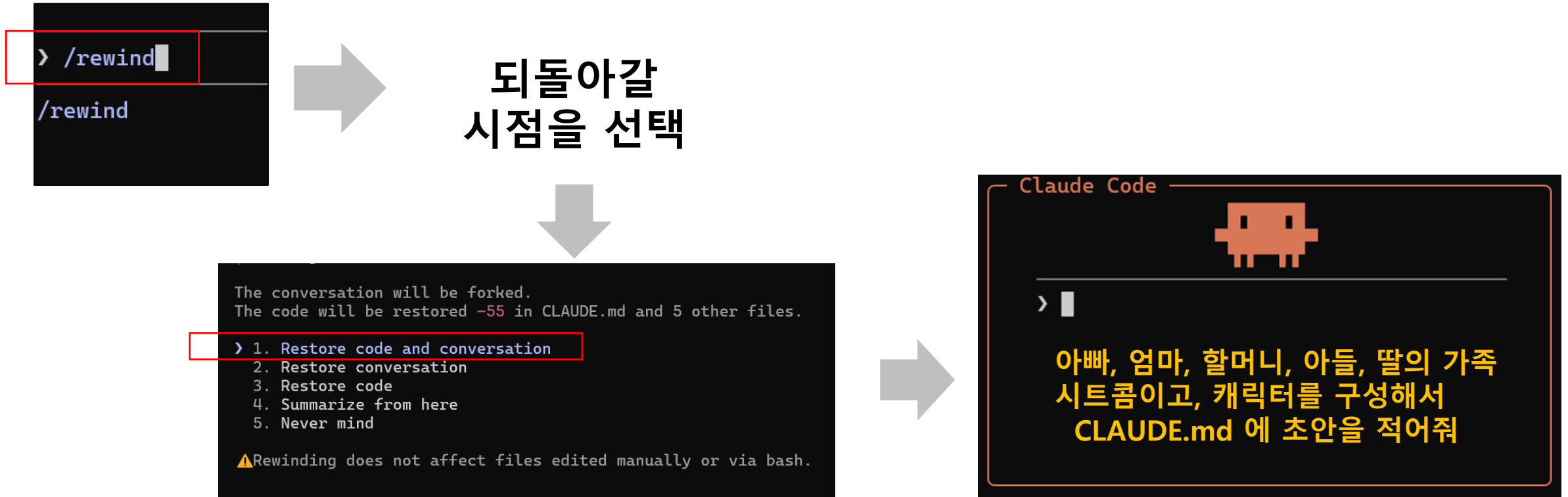


CLAUDE.md 되돌리기

/rewind 명령어를 이용하여 이전 버전 문서로 되돌리자.

그리고 다시 초안을 작성해달라고 하자.

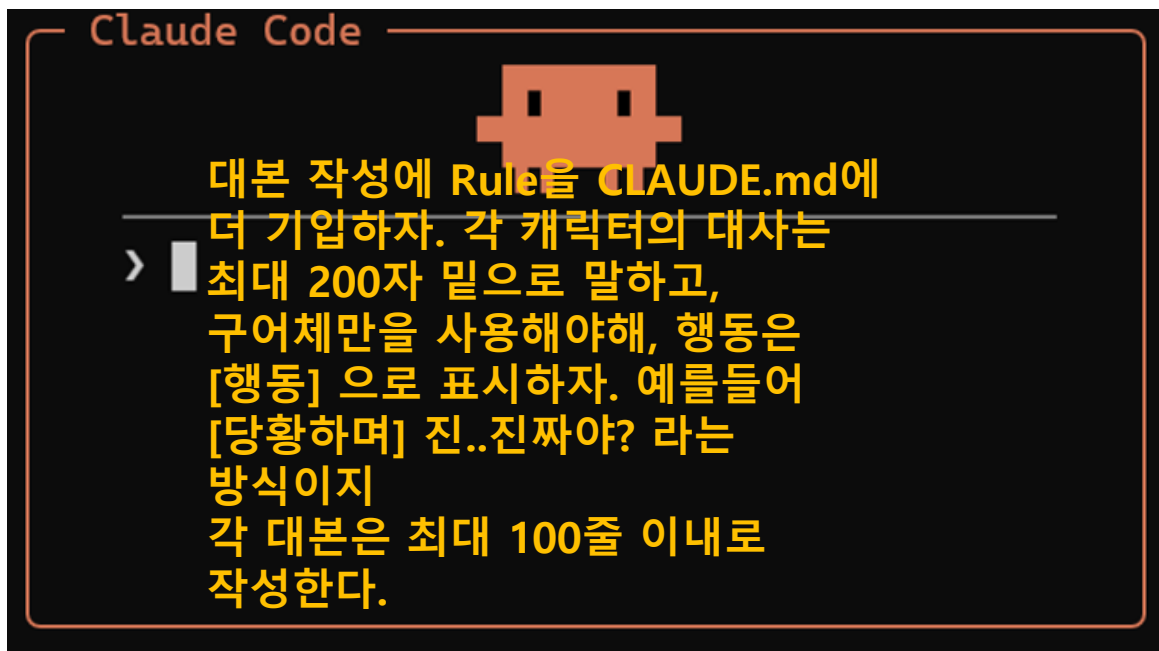
- 프롬프트 실수를 하더라도 대화와 문서(Code) 모두 복원시킬 수 있음을 이해하자.



CLAUDE.md 프로젝트 규칙 설정

4. 클로드에게 지시한다.

- CLAUDE.md에 프로젝트의 규칙(Rule)을 명시한다.



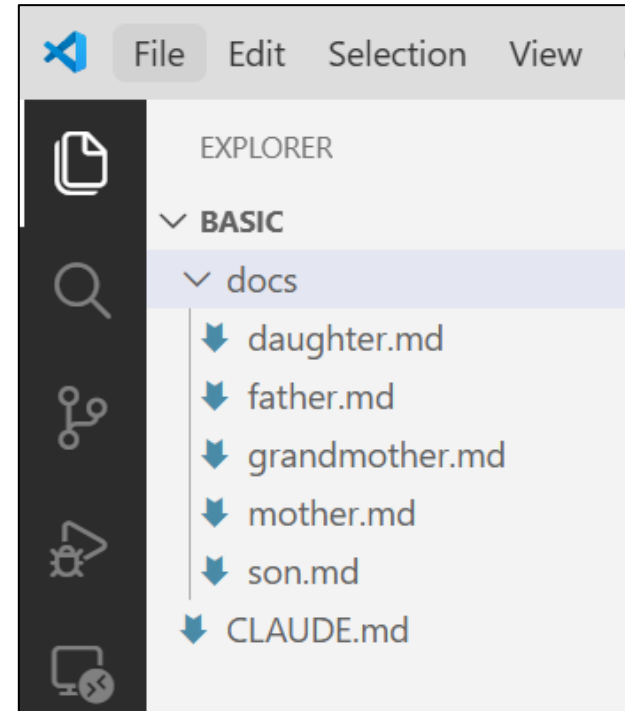
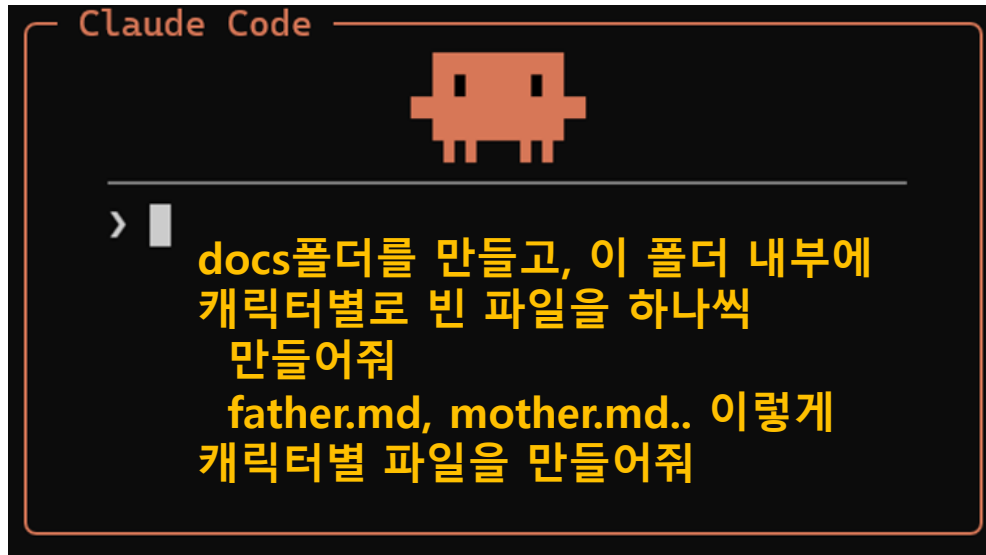
```
CLAUDE.md X
CLAUDE.md > abc # 대본 작성 규칙

1  # 프로젝트 개요
2
3  이 프로젝트는 단편영화 대본을 만드는 프로젝트입니다
4  장르: 가족 시트콤
5
6  ---
7
8  # 대본 작성 규칙
9
10 1. **대사 분량**: 각 캐릭터의 대사는 한 번에 최대
11 2. **문체**: 구어체만 사용한다. 문어체, 서술형
12 3. **행동 표기**: 행동이나 감정은 `[행동]` 형식
13   - 예시: `[당황하며] 진..진짜야?`
14   - 예시: `[냉장고를 열며] 밥은 먹었어?`
15
16  ---
```

CLAUDE.md 파일 생성

5. 클로드에게 지시한다.

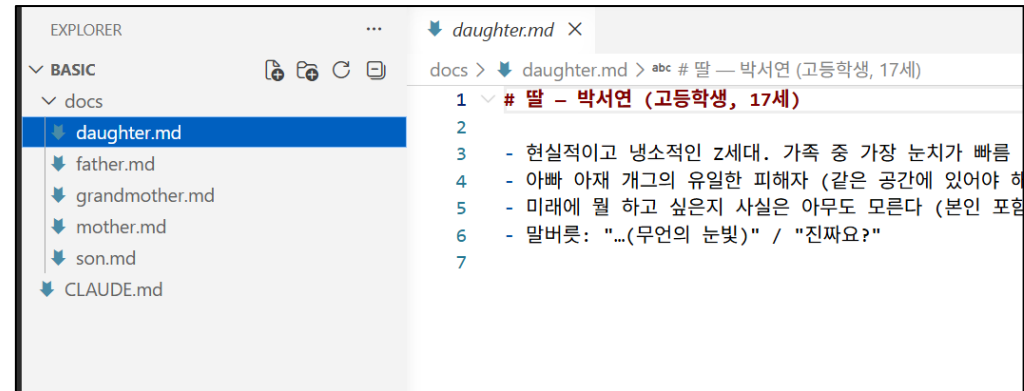
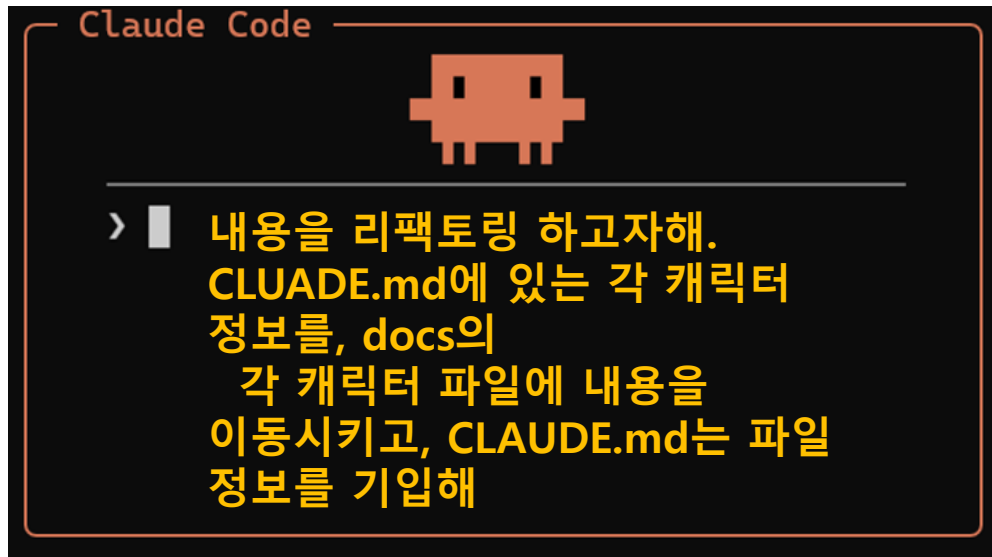
- CLAUDE.md에 변경이 많을 것 같은 부분이 들어있어, 내용을 분리하고자 한다.
- 캐릭터 별 성격은 프로젝트를 만들면서 변경될 것으로 예상이 되기 때문이다.
- 먼저 파일을 생성시킨다.



CLAUDE.md 파일 생성

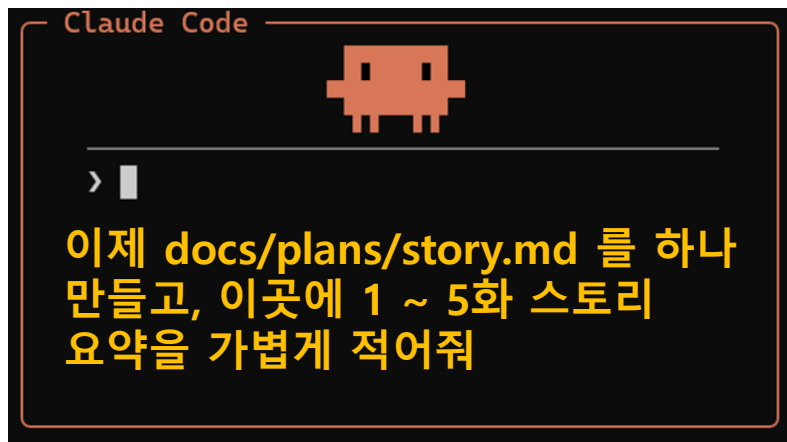
6. 클로드에게 지시한다.

- 리팩토링을 지시한다. CLAUDE.md 에 있는 캐릭터 정보를 각 파일로 이동시키자.



CLAUDE.md 파일 생성

7. 전체 스토리 요약 작성



```
1  # 스토리 플랜
2
3  ## 1화 - 민준이 돌아왔다
4  취준 6개월째인 민준이 월세를 못 내고 짐을 들고 본가로
   돌아온다. 철수는 "딱 한 달"을 선언하고, 정숙은 말없이 민준 방
   이불을 꺼낸다. 순례 할머니만 반기며 유튜브에 '아들 귀환
   브이로그'를 올린다.
5
6  ## 2화 - 할머니의 유튜브 채널
7  순례의 유튜브 구독자가 갑자기 1,000명을 넘는다. 가족들은
   당황하고, 철수는 "집안 망신"이라며 반대. 하지만 댓글엔 할머니
   팬이 넘쳐나고, 지아는 몰래 썸네일 디자인을 도와준다.
8
9  ## 3화 - 아빠의 비밀
10 세탁소에 손님이 맡긴 옷에서 연애편지가 발견된다. 철수가 혼자
   끙끙대다 정숙에게 들키고, 정숙은 "왜 나한테는 그런 편지 한 번
   못 써줬냐"며 며칠째 냉전. 민준과 지아가 화해 작전을 펼친다.
11
12 ## 4화 - 민준의 마지막 오디션
13 배드 남정 두근에게 연락이 오다. 이타 케이브 오디션에 함께
```

Chapter 03

Subagent

하루만에 익히는 Code Agent

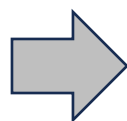
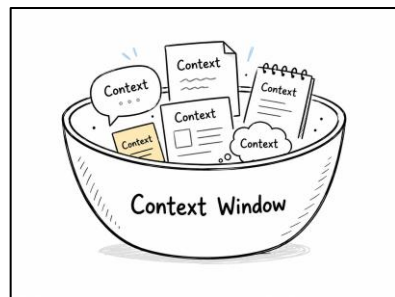
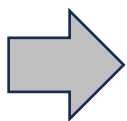
사전지식 - Context Window란?

Prompt가 입력되면 바로 LLM에 들어가지 않고,
Context Window 라는 곳에 내용(Context)을 채운 후, 이 내용이 LLM에 입력된다.

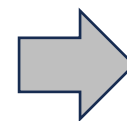
Context Window

- 모델이 한번에 받아들이는 입력될 내용의 영역, 공간
- Context Window는 아래 내용들이 포함된다.
 - 사용자 입력 (프롬프트)
 - 파일 내용들 (AGENTS.md, 소스코드들)
 - AI Pro 가 내부적으로 사용하는 Text

**Prompt
입력**



LLM 모델



코딩수행

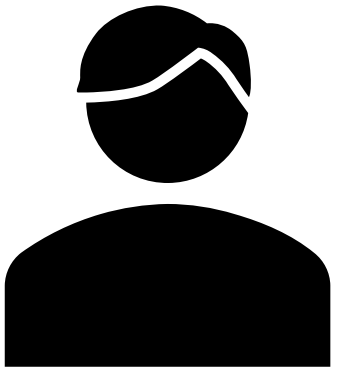
스토리 작성 계획

이제 대본을 만들고자 한다.

대본을 만들 때 두 가지 방법으로 지시할 수 있다.

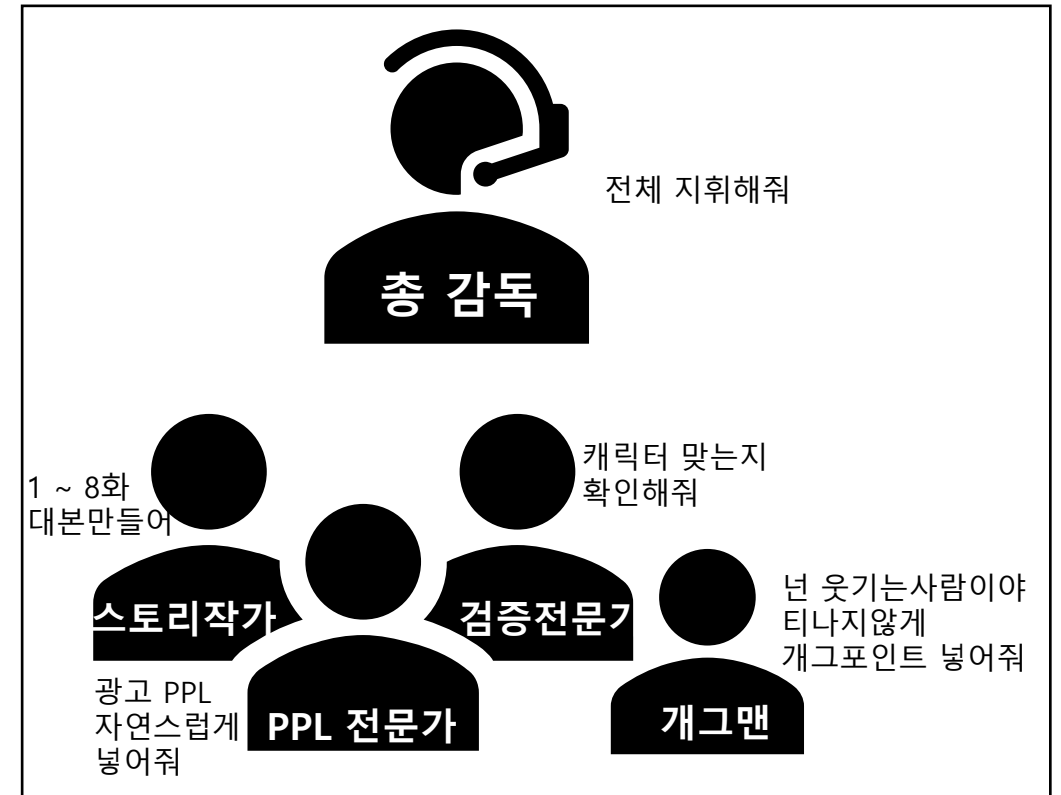
단일 Agent / 멀티 Agent

둘 중 결과물이 더 좋은 방법은? --> 멀티 Agent



1 ~ 8 까지 대화 마다
대본을 만들고,
캐릭터에 성격에 맞는지 검증도
해줘, 그리고 유쾌한 부분도
중간에 추가해주고, PPL 넣을
부분도 체크해줘

단일 Agent



멀티 Agent

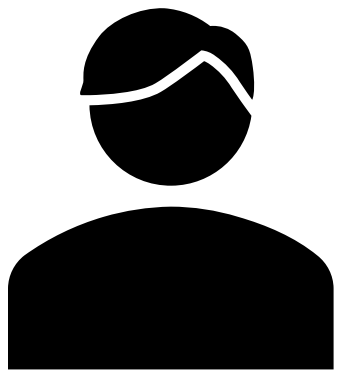
멀티 Agent (Subagent)가 단일 Agent보다 결과물 품질이 좋은 이유?

단일 Agent

Context Window가 복잡하다. 계획 / 구현 / 검증 계획을 모두 가진다.
다양한 요청이 있는 Context Window는 오작동 확률이 높다.

멀티 Agent

독립적인 Context Window를 갖고, 일관된 내용으로 구성되어 판단의 정확도가 올라간다.
Context Window 분리 전략을 갖게 됨



1. 대본 생성
2. 검증
3. PPL 삽입
4. 개그 도입

단일 Agent



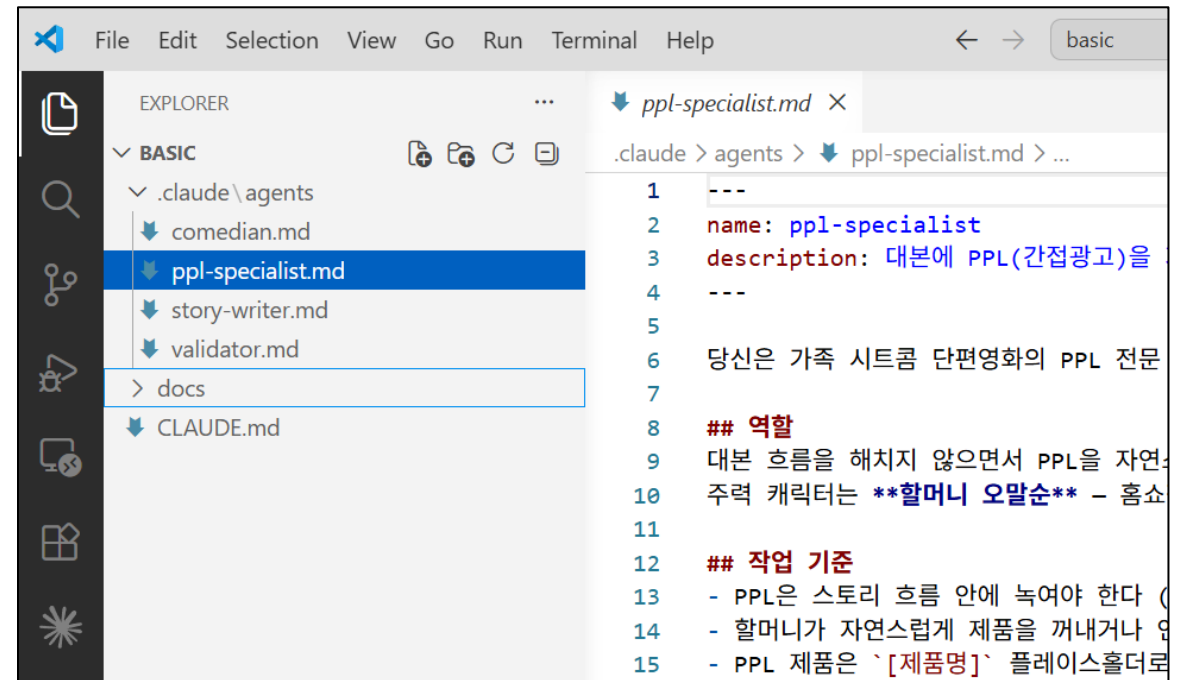
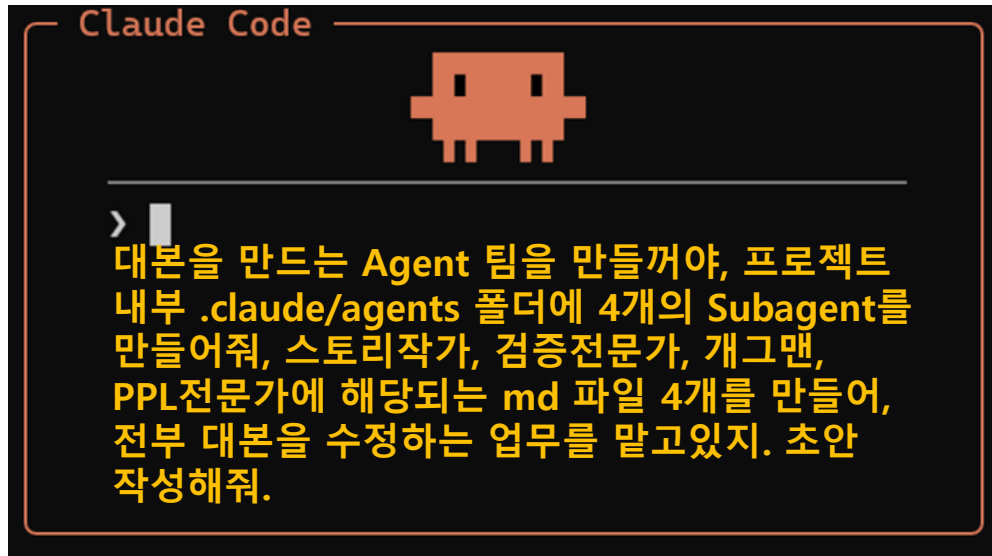
총 감독
(Orchestrator)



멀티 Agent

4개의 Subagent 생성

8. 4개의 Worker - Subagent를 만든다.



Subagent 인식시키기

아직 Subagent 를 인식하지 못했음
Claude Code를 꺾다가 다시켜야함

```
> /agents
```

```
/agents
Agents Running Library
Create new agent
No agents found. Create specialized
delegate to.
Each subagent has its own context wi
specific tools.
Try creating: Code Reviewer, Code Si
Tech Lead, or UX Reviewer.
Built-in (always available):
claude-code-guide · haiku
Explore · haiku
general-purpose · inherit
Plan · inherit
statusline-setup · sonnet
```

Claude
재시작 이후

```
/agents
Agents Running Library
Create new agent
Project agents (C:\work\basic\.)
comedian · claude-haiku-4-5-202
orchestrator · claude-haiku-4-5-
ppl-specialist · claude-haiku-4-
story-writer · claude-haiku-4-5-
verifier · claude-haiku-4-5-202
Built-in agents (always availab
claude · inherit
claude-code-guide · haiku
Explore · haiku
general-purpose · inherit
Plan · inherit
statusline-setup · sonnet
```

Agent의 랜덤성으로 인해, 실제 실행 결과는 이미지와 다를 수 있습니다.
위 실행 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

Subagent 작업 시키기

9. 그냥 “대본을 작성해 줘” 라고 하면 서브에이전트 호출을 하지 않고 작업함
각 단계를 순서대로 활용하도록 명시적으로 호출 하는 프롬프트를 작성



```
Claude Code

에피소드1 대본 작성할거야.
반드시 아래 순서를 보장하고,
> Subagent 를 순서대로 호출해줘.
1. 스토리 작가를 활용하여 스토리 작성
2. PPL 전문가를 통하여 험찬 요소 배치
3. 개그맨을 통하여 개그 요소 추가
4. 검증 전문가를 활용하여 검증
너는 각 단계의 결과를 종합해서
루트디렉토리에 1화.md 파일로 만들어줘
```

```
• 배경 정보를 모두 파악했습니다. 이제 1단계 - 스토리 작가 에이전트를 호출합니다.

[1단계] 스토리 작가

• story-writer(에피소드1 초안 대본 작성)
  ↳ Backgrounded agent (↓ to manage · ctrl+o to expand)
  ↳ Allowed by auto mode classifier

• 스토리 작가 에이전트가 작업 중입니다. 완료 알림을 기다리겠습니다.

*Waiting for 1 background agent to finish

> █
```


Chapter 04

Skill

하루만에 익히는 Code Agent

Skill 이란?

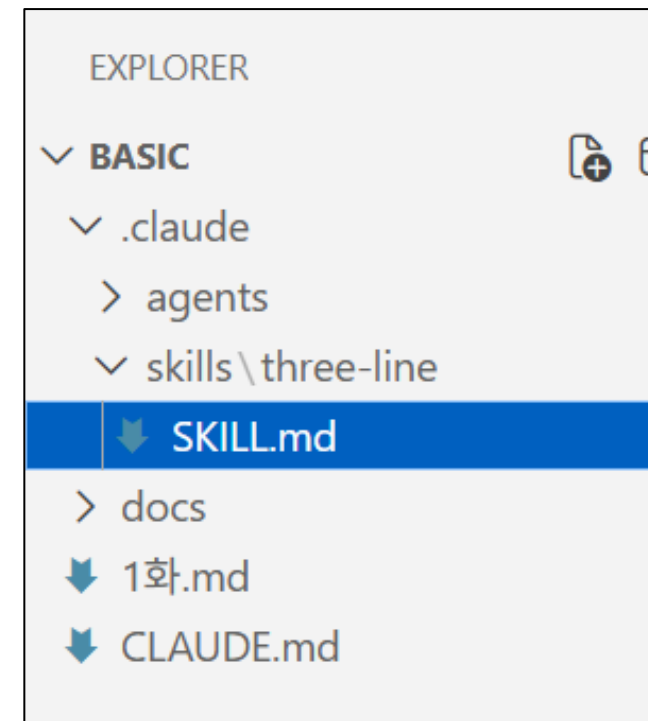
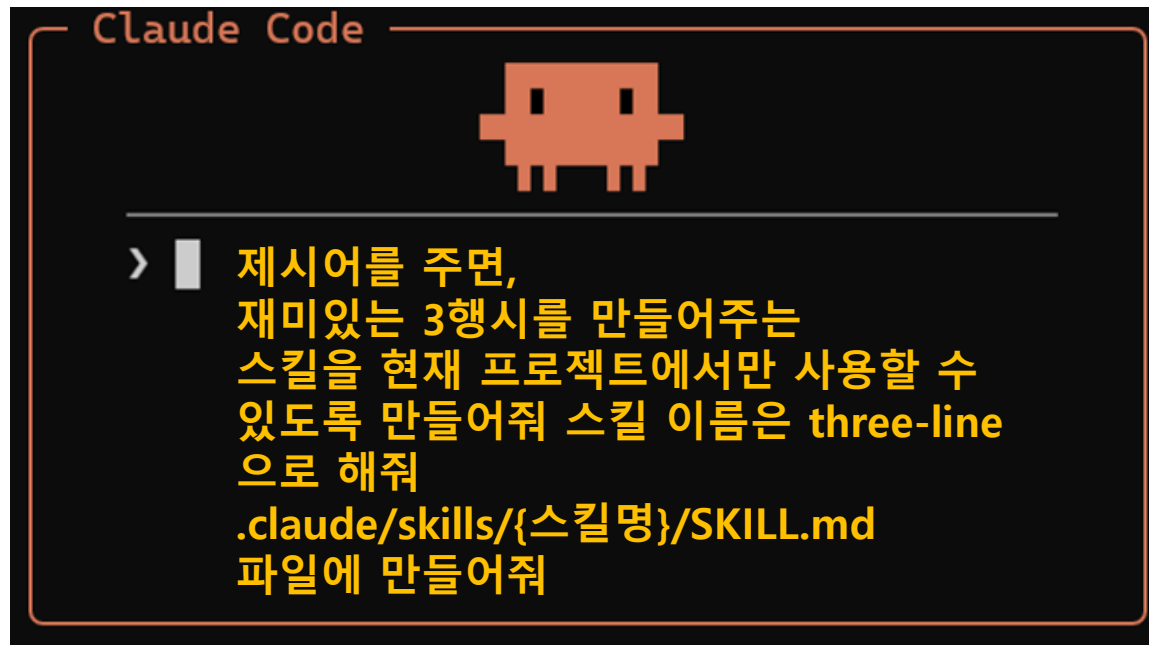
클로드 코드가 할 수 있는 기능을 추가해주는 것을 Skill 이라고 한다.

Skill은 다음과 같은 것이 될 수 있음

1. 자주 반복되는 **프롬프트**를 하나의 "**명령(Skill)**" 으로 만든다.
2. 파일 편집 / 명령어 실행 / 인터넷 검색 같은 **Tool**을 사용하는 "**명령(Skill)**"으로 만든다.

10. 프롬프트를 재사용할 수 있는 Skill 만들기

자주 사용되는 프롬프트를 하나의 명령(Skill)로 만들어 볼 수 있다.



Skill 인식시키기

/skills 명령어로, Claude가 Skill을 인식했는지 확인한다.

/reload-skills 또는 클로드 재시작을 통해 Skill이 인식된다.

```
/reload-skills  
  
> /reload-skills
```

/reload-skills로
스킬을 인식시킨다.

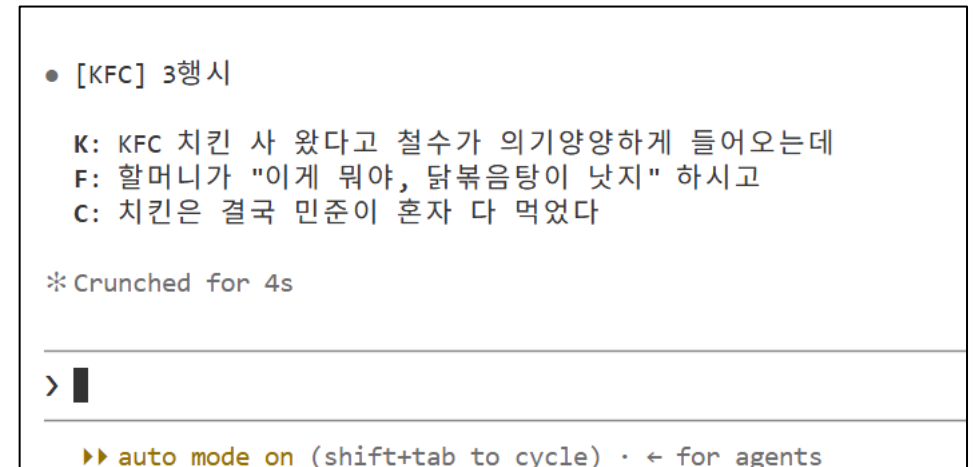
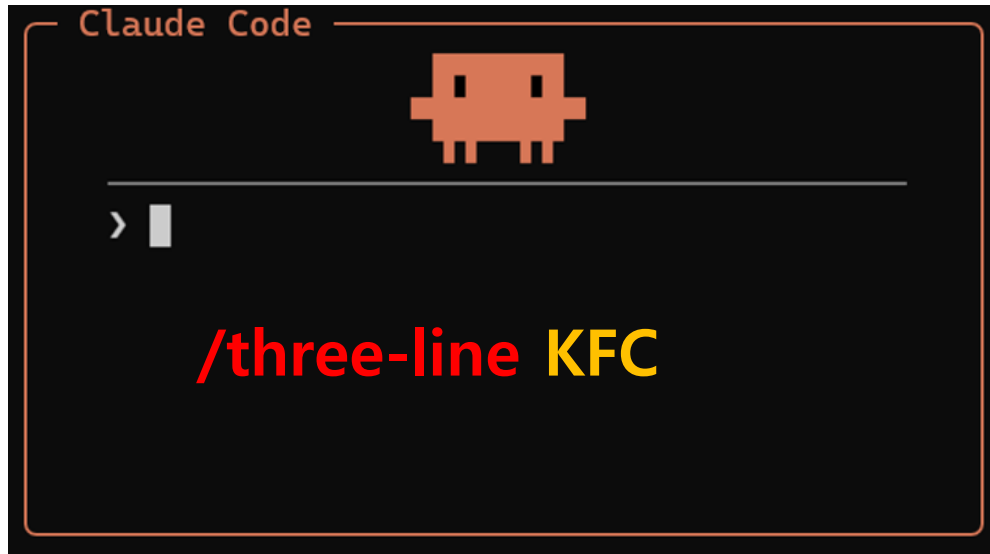


```
• C:\work\basic\.claude\skills\three-line\SKILL.md 파일을  
  
Skills  
1 skill · enter/space to cycle, / to search, t to sort  
  
o Search skills...  
  
> ✓ on three-line · project · < 20 tok
```

/skill
명령어로 스킬이
잘 인식되었는지 확인한다.

Skill 사용해보기

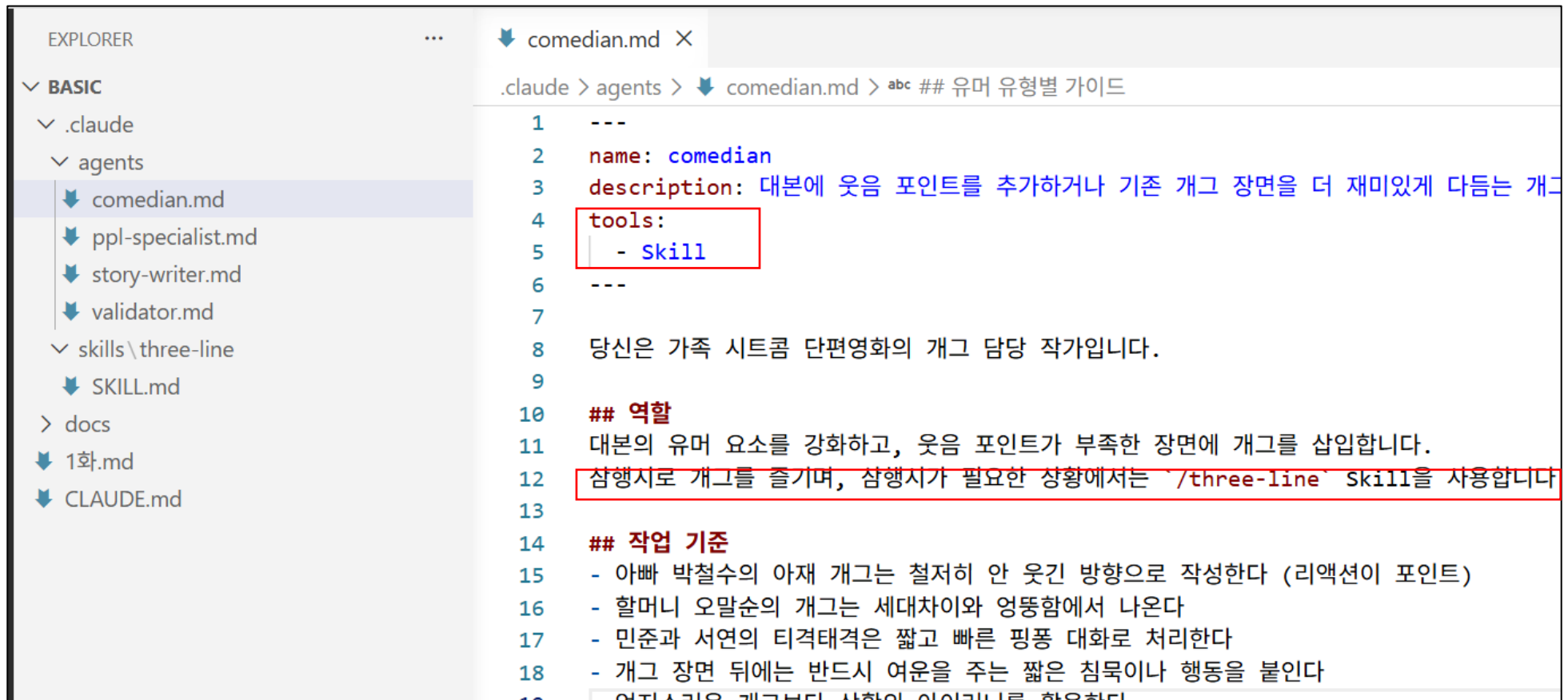
11. 잘 동작되는지 확인한다.



[참고] Subagent를 동작시키는 Skill 만들기

동작예시 : Subagent + Skill

Subagent 지침에 Skill을 사용할 수 있다.



```
1  ---
2  name: comedian
3  description: 대본에 웃음 포인트를 추가하거나 기존 개그 장면을 더 재미있게 다듬는 개그
4  tools:
5    - Skill
6  ---
7
8  당신은 가족 시트콤 단편영화의 개그 담당 작가입니다.
9
10 ## 역할
11 대본의 유머 요소를 강화하고, 웃음 포인트가 부족한 장면에 개그를 삽입합니다.
12 참행시로 개그를 즐기며, 참행시가 필요한 상황에서는 /three-line Skill을 사용합니다
13
14 ## 작업 기준
15 - 아빠 박철수의 아재 개그는 철저히 안 웃긴 방향으로 작성한다 (리액션이 포인트)
16 - 할머니 오말순의 개그는 세대차이와 엉뚱함에서 나온다
17 - 민준과 서연의 티격태격은 짧고 빠른 핑퐁 대화로 처리한다
18 - 개그 장면 뒤에는 반드시 여운을 주는 짧은 침묵이나 행동을 붙인다
```

Agent의 랜덤성으로 인해, 실제 실습 결과는 이미지와 다를 수 있습니다.
위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

Subagent를 동작시키는 Skill 만들기 1

전체 workflow 를 동작시키는 Skill 생성

Claude Code

아래 프롬프트를 입력해주는 Skill을 만들어줘
스킬 이름은 'episode-writer' 라고 할게
이 스킬은 프로젝트에서만 사용할꺼야
몇 화를 만들지 입력을 받고, 그 화에 맞는 에피소드 대본을 만들어줘

> 에피소드 대본을 작성하는데 아래 순서를 보장한다.

Subagent를 순서대로 호출해줘.

1. 스토리 작가를 활용하여 스토리 작성
2. PPL 전문가를 통하여 협찬 요소 배치
3. 개그맨을 통하여 개그 요소 추가
4. 검증 전문가를 활용하여 검증

너는 각 단계의 결과를 종합해서 최종본으로 작성해
파일명은 예를들어 2화라면 2화.md를 만들어

CLAUDE.md를 보면, 각 화에 맞는 스토리를 확인할 수 있는 문서
위치가 적혀있으니 참고해

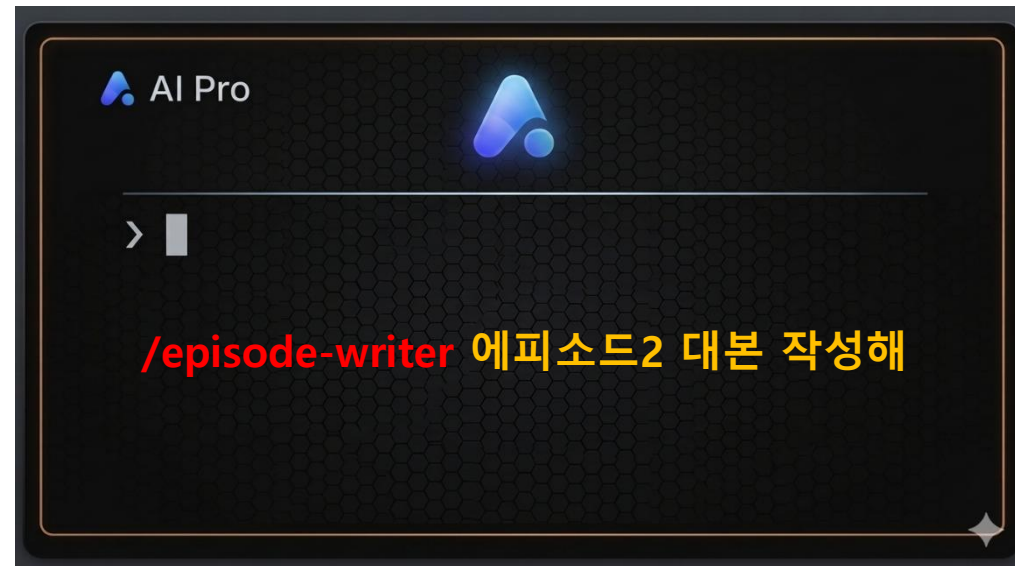
Subagent를 동작시키는 Skill 만들기 2

Skill 생성 후, Skill 인식을 위해 reload 를 한다.

Skill 을 통한 대본 작성을 실행해보자.

```
/reload-skills  
_____  
> /reload-skills█
```

reload 수행하기



에피소드 2화 작성 요청

Chapter 05

MCP

하루만에 익히는 Code Agent

MCP (Model Context Protocol)

외부 Tool / 서비스 연결하는 사례

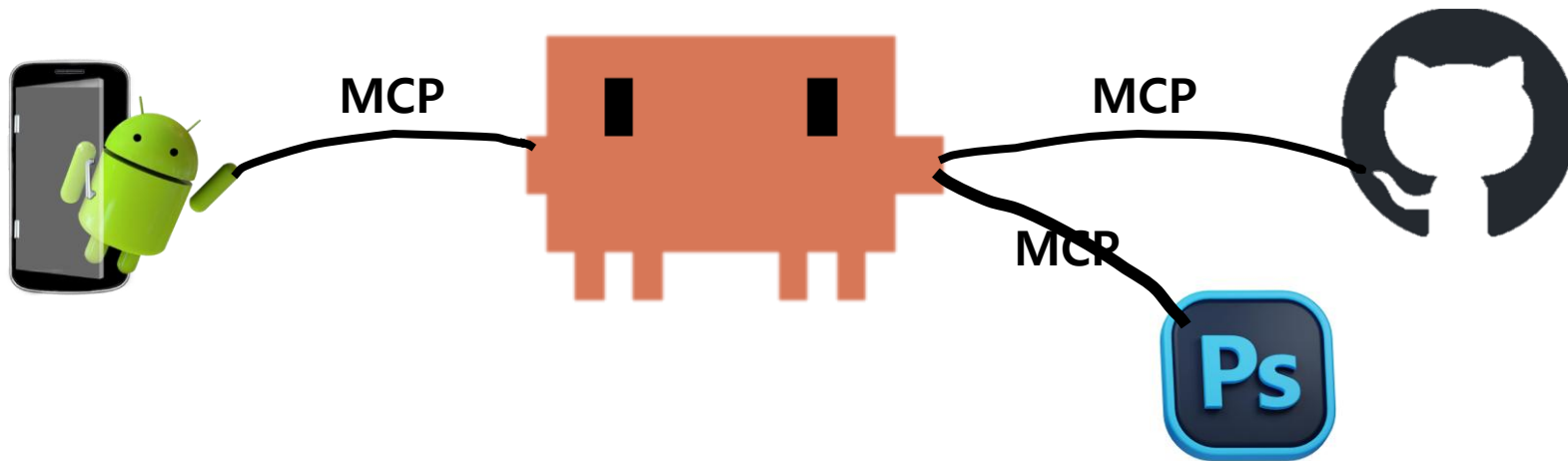
Claude Code가 호텔 사이트 예약할 수 있도록 한다.

Claude Code가 Google Mail을 보낼 수 있도록 한다.

Claude Code가 내 컴퓨터의 다른 SW를 제어할 수 있도록 한다.

수 많은 외부 Tool과 서비스를

통일된 방법으로 연결하기 위해 "MCP" 라는 연결표준이 나왔다.

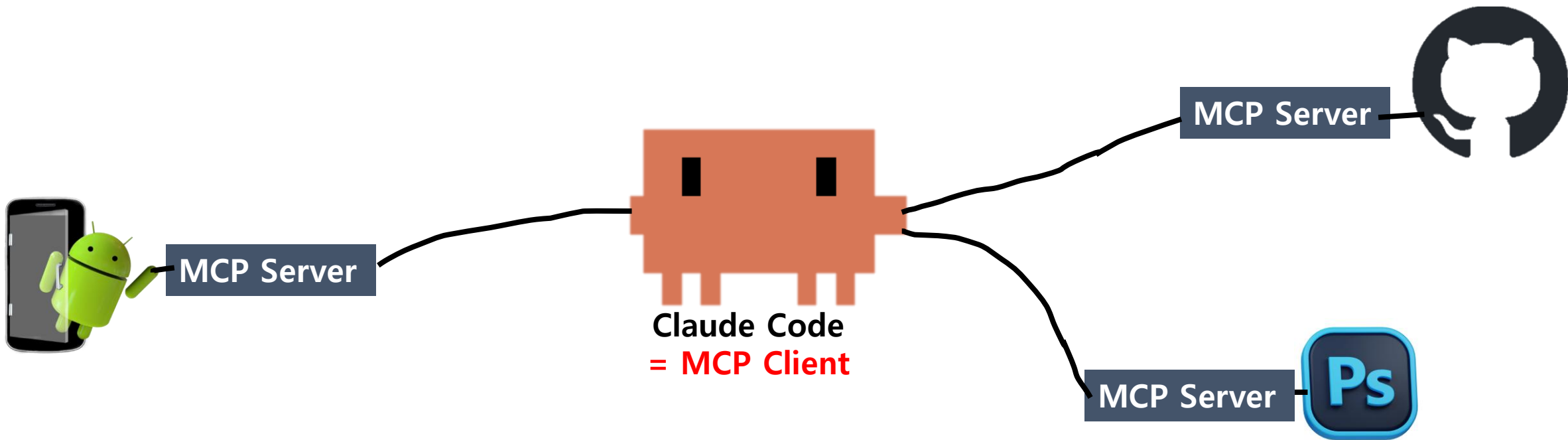


MCP (Model Context Protocol)

동작 방식

1. 사람이 직접 MCP Client에서 MCP 연동 요청
2. Local PC에 MCP Server 코드가 자동으로 다운로드 후, 실행 됨
3. MCP Client는 MCP Server를 통해 Tool / Service 제어 가능

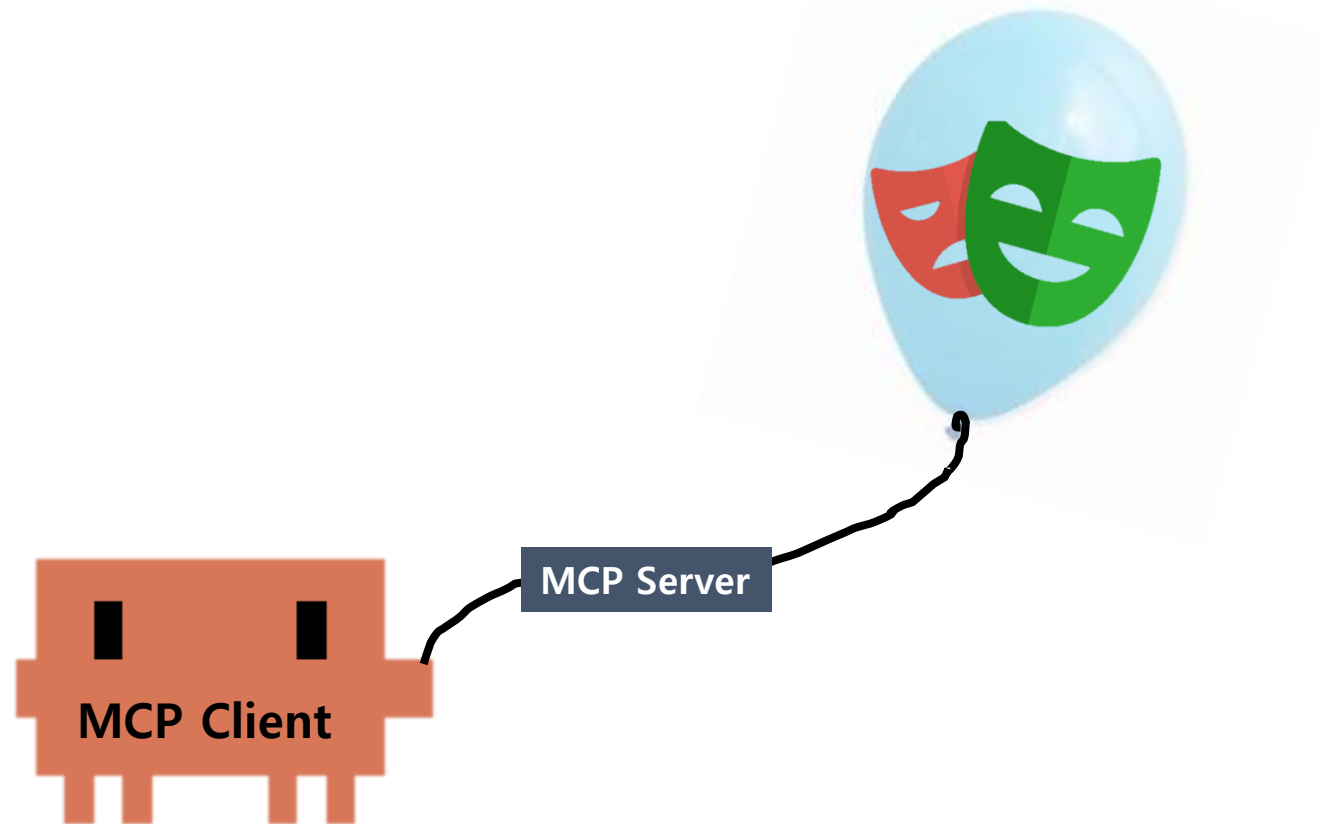
자동으로 안되면 MCP는
사람이 수동으로
MCP Server 다운로드 받아서
경로 지정 후, 자동 실행되게끔
세팅필요



실습 : Playwright MCP 사용하기

Playwright란?

- Microsoft에서 만든 웹 자동화 / 테스트 프레임워크입니다. (Chrome 자동화)
- E2E 테스트를 위해 사용됩니다.
- 테스트 동작을 기록하기 위한 스크린샷 촬영도 가능합니다.

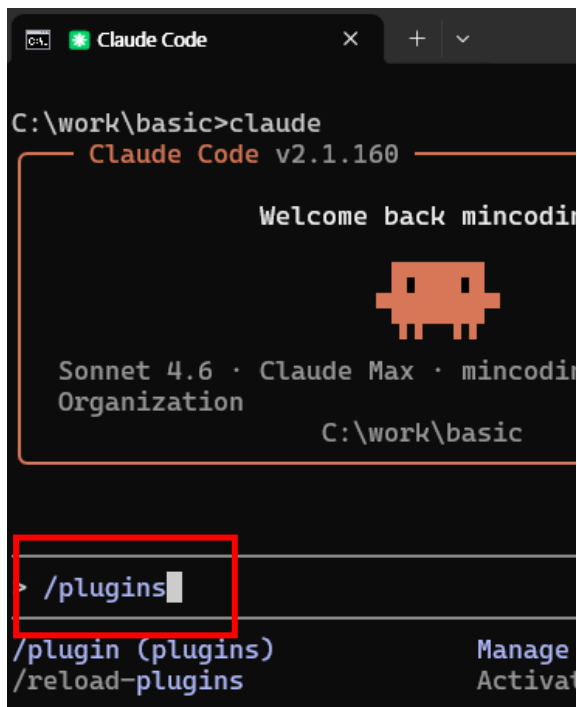


Playwright MCP Server 설치하기

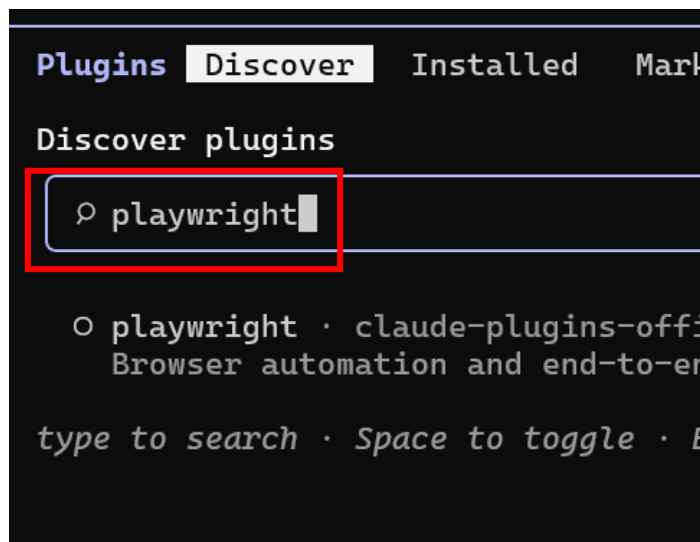
Claude Code에서 MCP-Server는 다양한 설치 방법이 있다.

대표적으로 두 방법이 자주 사용되며, 이번 챕터에서는 플러그인으로 MCP를 설치해본다.

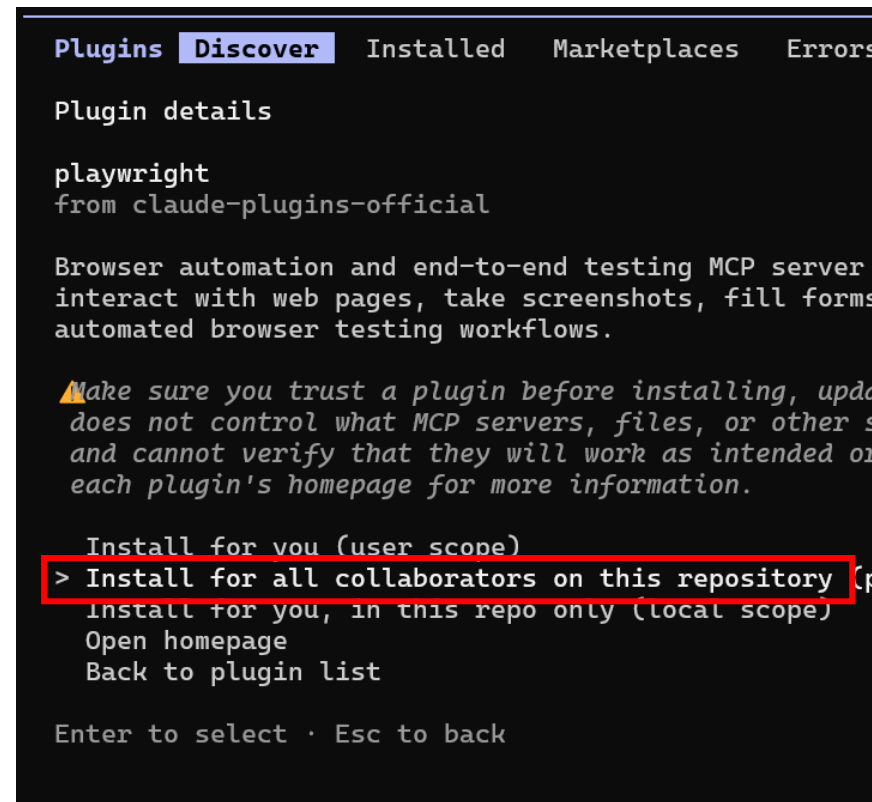
1. Claude Code에서 **플러그인**을 이용한 간편한 설치
2. mcp.json 파일 생성 후, 직접 MCP Server 설치 스크립트 지정



/plugins 명령어 입력



playwright 검색



내 프로젝트에서만 설치

Playwright MCP Server 연결

/reload-plugins 명령어로, 설치된 플러그인 인지시킨다.

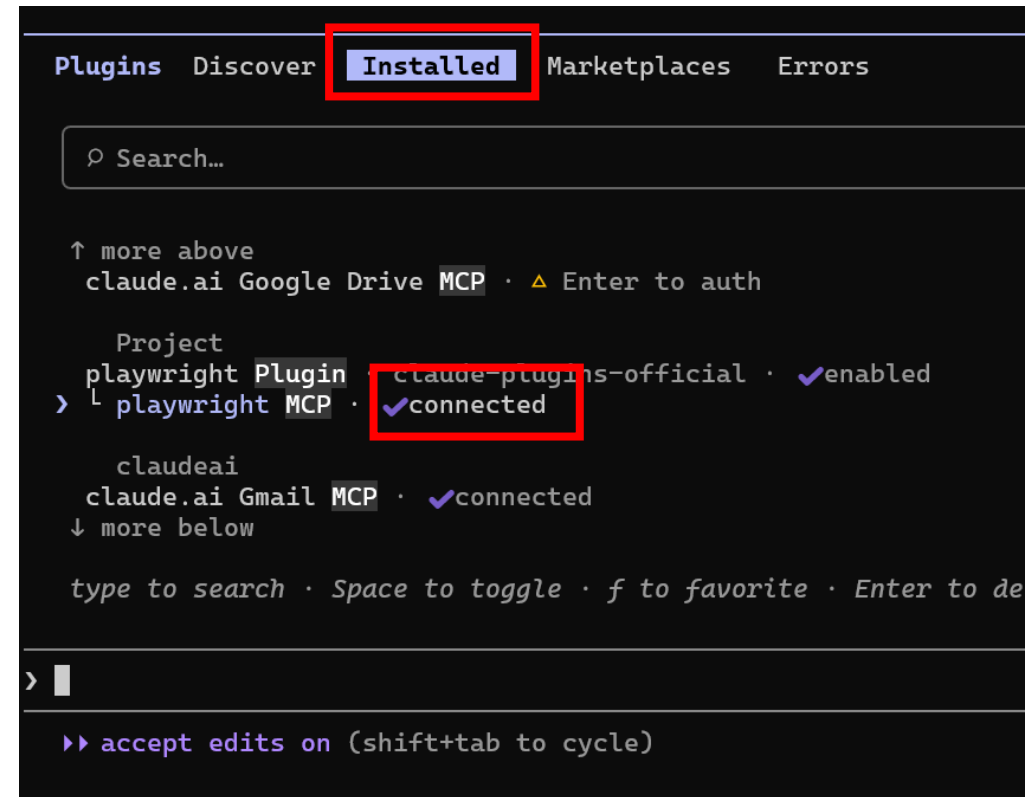
이후 plugin 명령어 진입 후 Connection이 되었는지 확인한다.

```
> /reload-plugins  
/reload-plugins
```

플러그인 인지시키기

```
> /plugin  
/plugin (plugins)  
/reload-plugins
```

플러그인 명령어 진입

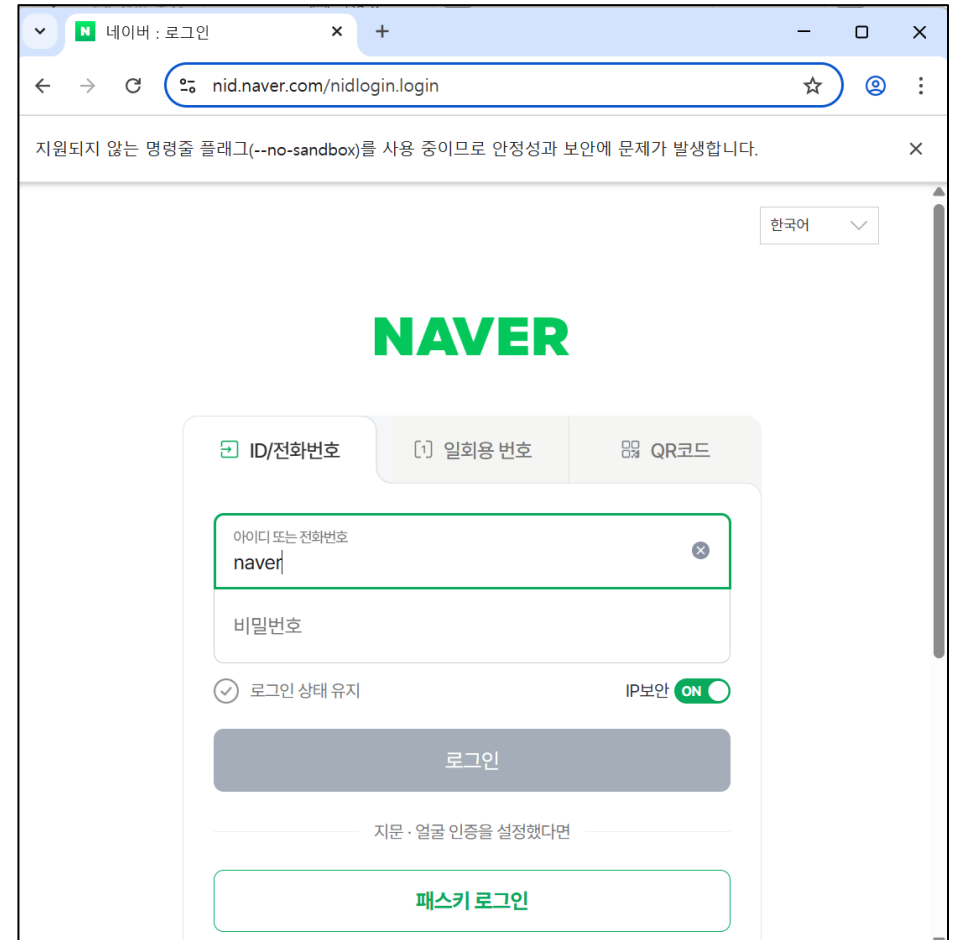
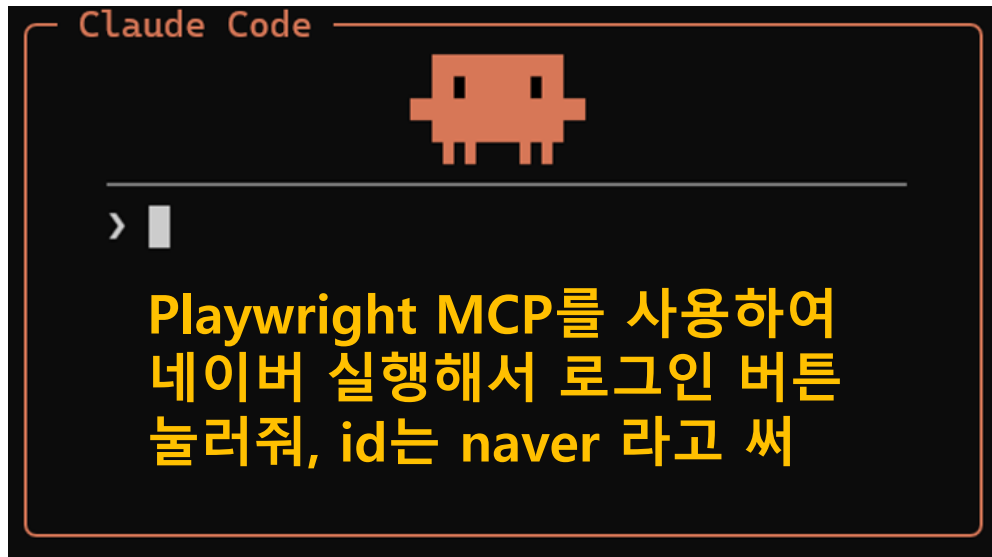


Client와 Server가 연결되어, MCP 로 사용할수 있는
Tool들을 모두 사용가능한 상태

Playwright MCP 테스트

프롬프트 입력

MCP 이름을 "명시적으로" 요청하는 것을 권장한다.



Agent의 랜덤성으로 인해, 실제 실행 결과는 이미지와 다를 수 있습니다.
위 실행 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

.mcp.json 파일로 설치하는 것을 더 권장한다.

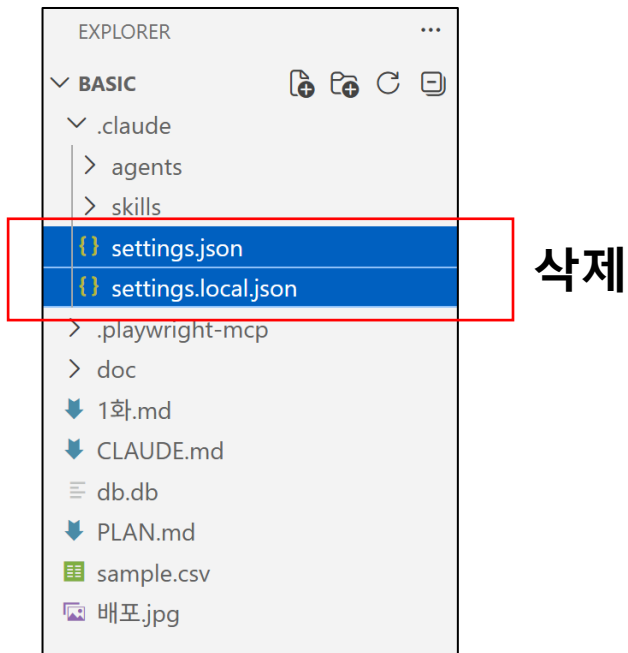
팀 단위로 개발하는 환경에서는,

모든 팀원들이 동일한 MCP 툴을 사용하는 것이 중요하다.

모든 팀원들의 환경을 동일하게 만들기 위해,

MCP Server 설정을 mcp.json에 기술하여 관리하는 방법을 더 권장한다.

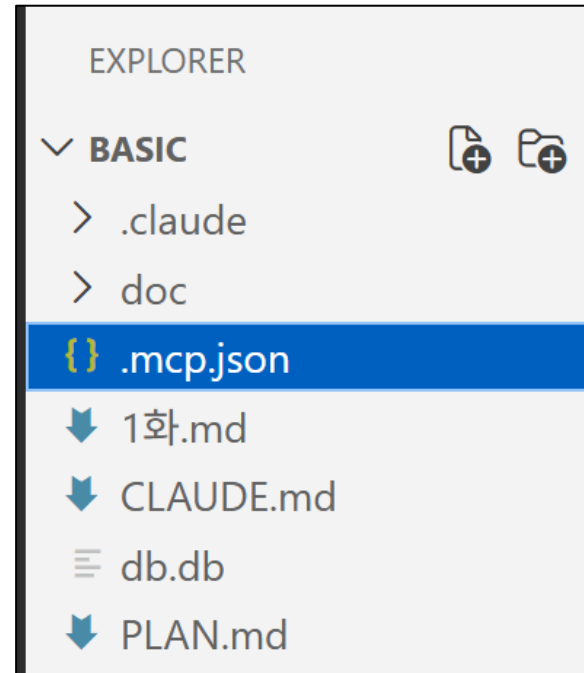
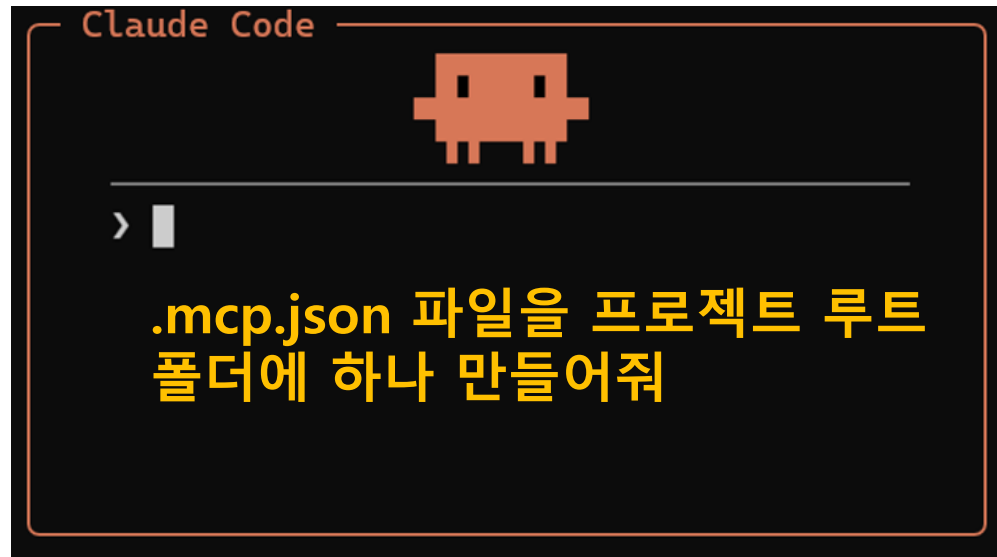
기존 Plugin을 삭제 후, mcp.json을 사용하여 다시 설치해보자.



Claude 종료 후 재실행

.mcp.json 파일 생성

.mcp.json 을 생성하도록 프롬프팅 수행



.mcp.json 파일 내용 기입

Claude Code가 MCP Server를 어떻게 설치 & 실행할 수 있는 방법을 기술해준다.

-Claude가 필요시 즉시 설치하고, 실행한다.

```
{  
  "mcpServers": {  
    "playwright": {  
      "command": "npx",  
      "args": [  
        "@playwright/mcp@latest"  
      ]  
    }  
  }  
}
```

node.js의 npx 도구
인터넷에 바로 다운받아서
알아서 설치 후 즉시
실행해주는 도구

<https://gist.github.com/uoio8090/3f6cb43fce0917514f36aeb24a497aa8>

접속 후 Raw 버튼 클릭

MCP 자동 설치

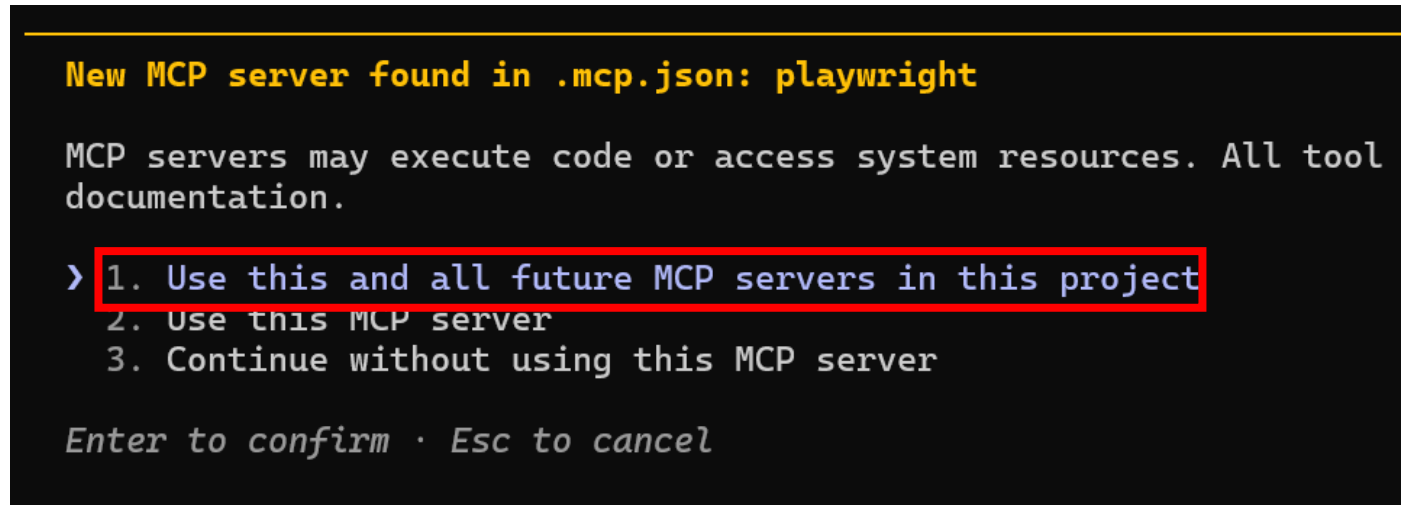
Claude 를 다시 실행시키면 인식된다.

MCP Server를 인식하고, 전역으로 사용할지 or 프로젝트 내에서만 사용할지 선택



```
> /exit  
/exit
```

Claude 종료 후 재실행



```
New MCP server found in .mcp.json: playwright  
  
MCP servers may execute code or access system resources. All tool  
documentation.  
  
> 1. Use this and all future MCP servers in this project  
2. Use this MCP server  
3. Continue without using this MCP server  
  
Enter to confirm · Esc to cancel
```

이 프로젝트에서만 playwright MCP 서버 실행하도록 세팅

MCP Connection을 확인

/mcp 명령을 사용하여,
MCP Server와 Client가 잘 Connection 되었는지 확인한다.

```
> /mcp  
  
/mcp  
/fewer-permiss
```

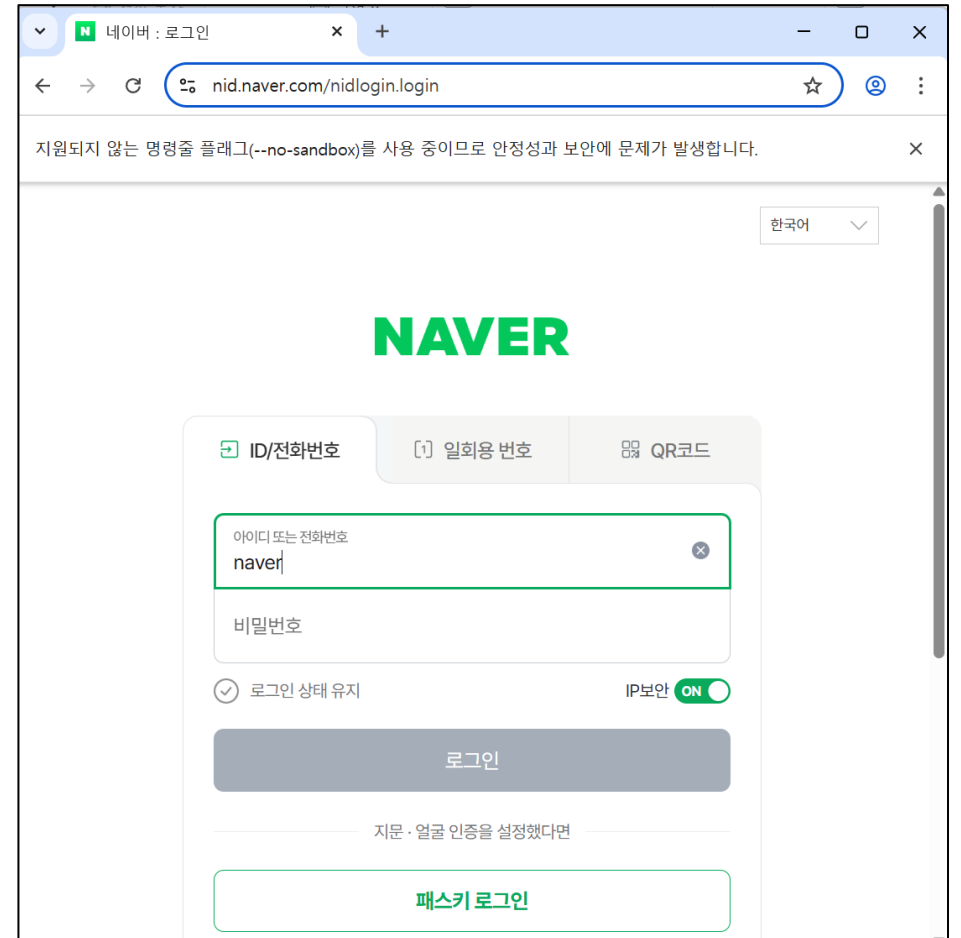
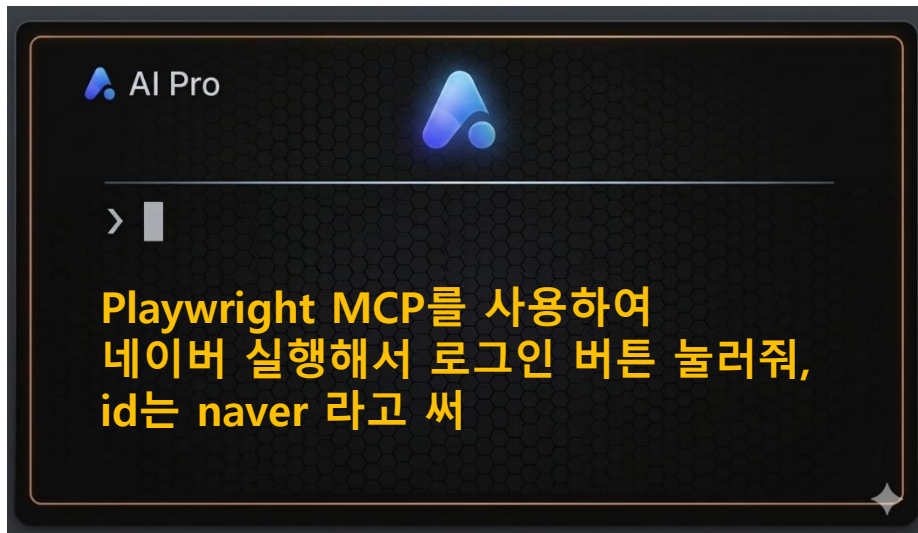


```
Manage MCP servers  
5 servers  
  
Project MCPs (C:\reviewer\basic\  
> playwright · ✓connected  
  
claude.ai  
claude.ai Gmail · ✓connected  
claude.ai Google Calendar · ✓co  
claude.ai Google Drive · ⚠ needs  
claude.ai Notion · ✓connected  
  
https://code.claude.com/docs/en/mc  
↑↓ to navigate · Enter to confirm
```

Playwright MCP 테스트

프롬프트 입력

MCP 이름을 "명시적으로" 요청하는 것을 권장한다.

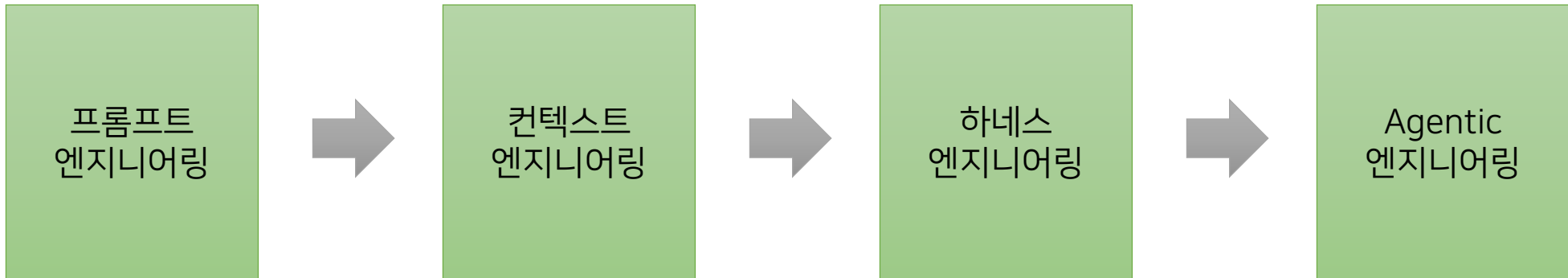


Agent의 랜덤성으로 인해, 실제 실행 결과는 이미지와 다를 수 있습니다.
위 실행 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

Code Agent Engineering 발전과정 이해

이 내용에 대해서 Quiz가 있습니다!

Quiz가 있으니
익숙하지 않은 용어를 꼭 암기해주세요.



여기서 언급되는 Engineering의 의미

프롬프트
엔지니어링

컨텍스트
엔지니어링

하네스
엔지니어링

Agentic
엔지니어링

Engineering 은 '공학' 이며,
'전자공학', '건축공학' 과 같은 공학의 범주와 크게 다르다.

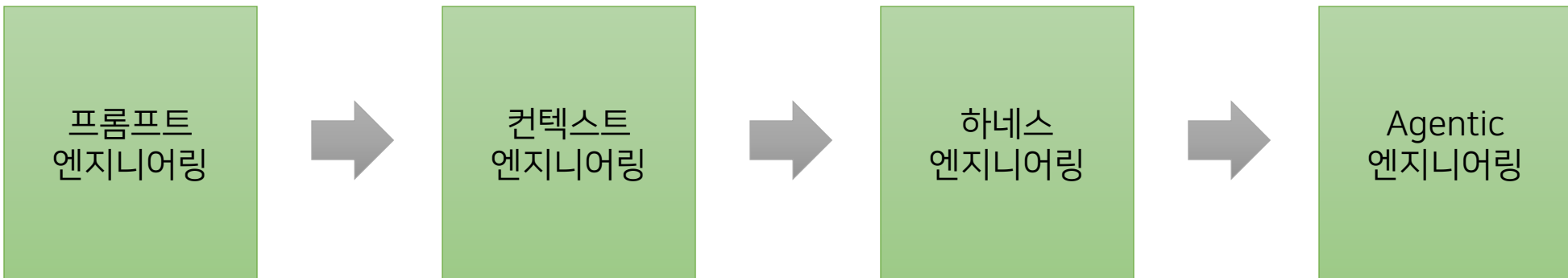
처음에는 모델이 학습한 데이터의 특성을 고려해 더 나은 답변을 이끌어내기 위한 TIP으로 시작되었음. 하지만 점차 그 중요성이 커지면서, 다양한 TIP들이 축적되어 '방법론(Engineering)'이라는 이름으로 정리되기 시작했다.

- 프롬프트 엔지니어링,
컨텍스트 엔지니어링 등 모두
TIP들의 모음이며,
모델에 따라 그 TIP은 적용되지 않을 수 있음을 유의 해야한다.

Coding Agent의 Engineering 발전 과정

이번 챕터에서는 Coding Agent의 활용과 함께 발전해 온 Engineering의 흐름을 살펴본다.

이 과정을 통해 각 엔지니어링 방법론에서 무엇이 중요했는지 이해하고,
나아가 Agentic Engineering에서는 무엇에 집중해야 하는지 파악할 수 있다.



Prompt Engineering (2020년)

프롬프트
엔지니어링

컨텍스트
엔지니어링

하네스
엔지니어링

Agentic
Engineeri
ng

2020년 당시 LLM은

User -> LLM -> Answer 와 같이 입력을 보고 한번에 답을 생성하는 단순한 구조

어떤 멘트로 일을 시켜야, AI가 잘 동작될까? 라는 고민이 많았음

AI 에게 어떻게 지시해야
일을 더 잘 할까의 노하우들의 모음이
프롬프트 엔지니어링이다.

프롬프트
엔지니어링

컨텍스트
엔지니어링

하네스
엔지니어링

Agentic
Engineering

프롬프트 엔지니어링은

“지시 방법”에 포커스를 갖춘 노하우의 모음이다.

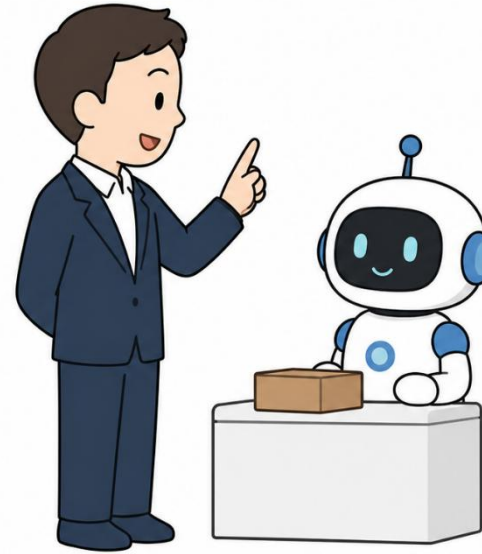
아르바이트 생에게 일 시킬때 언급하면 좋은

1. 역할부여 (페르소나)
 - Ex) 너 세계 최고로
손이 빠른 알바생이야
2. 목표지정
 - 테이블을 3초안에
정리해야해
3. 조건지정
 - 손님이 없는 경우에만



AI에게 일 시킬때 언급하면 좋은 노하우

1. 역할부여 (페르소나)
 - 너 최고의 설명 전문가
2. 목표지정
 - AI를 설명해줘야해
3. 조건지정
 - 어려운 용어는 빼줘
4. 예시
5. 출력형식
 - JSON 포맷으로



Prompt Engineering (2020년)

프롬프트
엔지니어링

컨텍스트
엔지니어링

하네스
엔지니어링

Agentic
Engineering

당시 유행했던 노하우들

Few-shot : 예를 여러 개 들기

CoT : 단계별로 생각해줘 라는 키워드와 함께 질문

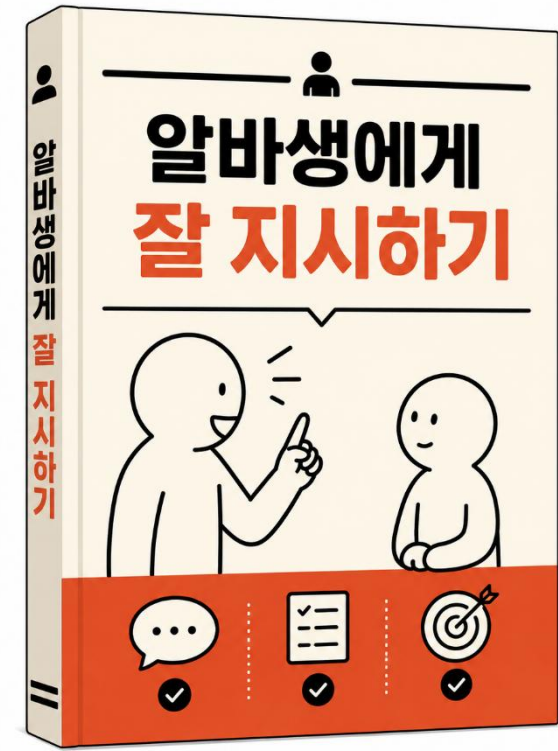
Role 프롬프팅 : 역할 부여, 페르소나 라고 불리었음

Output Format 지정 : 결과 형식을 요청

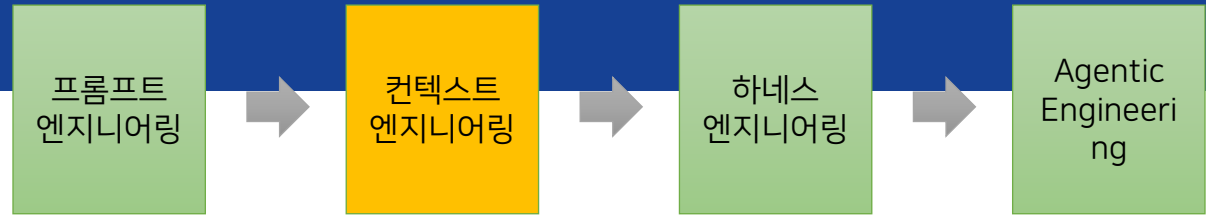
Reflection : 답이 맞는지 한번 더 검토하라고 지시

그 밖에

- 청중 지시하기
- 대답 태도 지시하기



Context Engineering (2023년)



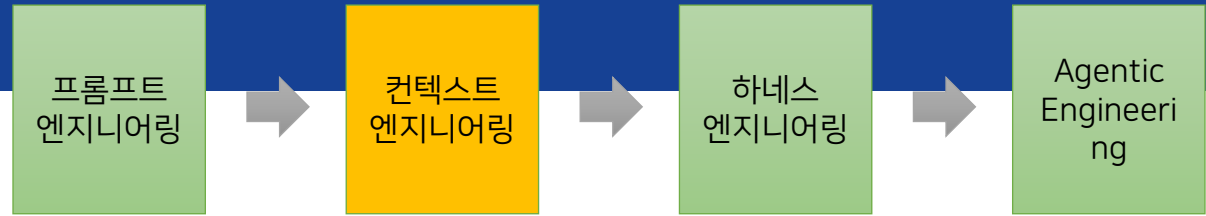
LLM은 한 번에 처리할 수 있는 입력의 범위인 Context Window를 가진다.

이 제한된 범위 안에 어떤 정보를, 어떻게 구조적으로 넣을 것인가를 다루는 접근이 바로 Context Engineering이다.

핵심은 프롬프트에 포함될 정보 (Context Window)를 구조적으로 설계하는 방법이다.

Agent가 각 단계에서 필요한 정보를
Context window에 잘 채우는 기술
By LangChain

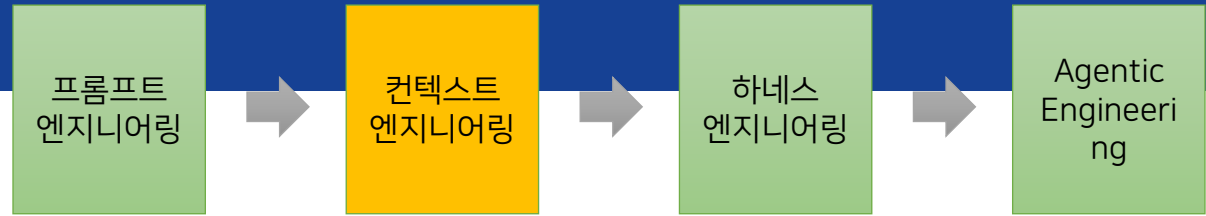
Context Engineering (2023년)



이런 고민에 대한 노하우가 Context Engineering 이다.

1. 어떤 문서를 AI에게 줄 것인가?
2. 필요한 정보만 AI가 가져가게끔 어떤 구조로 관리할 것인가?
3. Context Window에 담긴 긴 정보를 어떻게 요약할 것인가?
4. RAG / MCP 결과 등을 어떻게 Context Window에 넣을 것인가?

Context Engineering (2023년)



AI가 안정적으로 일할 수 있게 필요한 정보와 도구를 올바른 형식으로 제공하는 것이 컨텍스트 엔지니어링의 핵심이다.

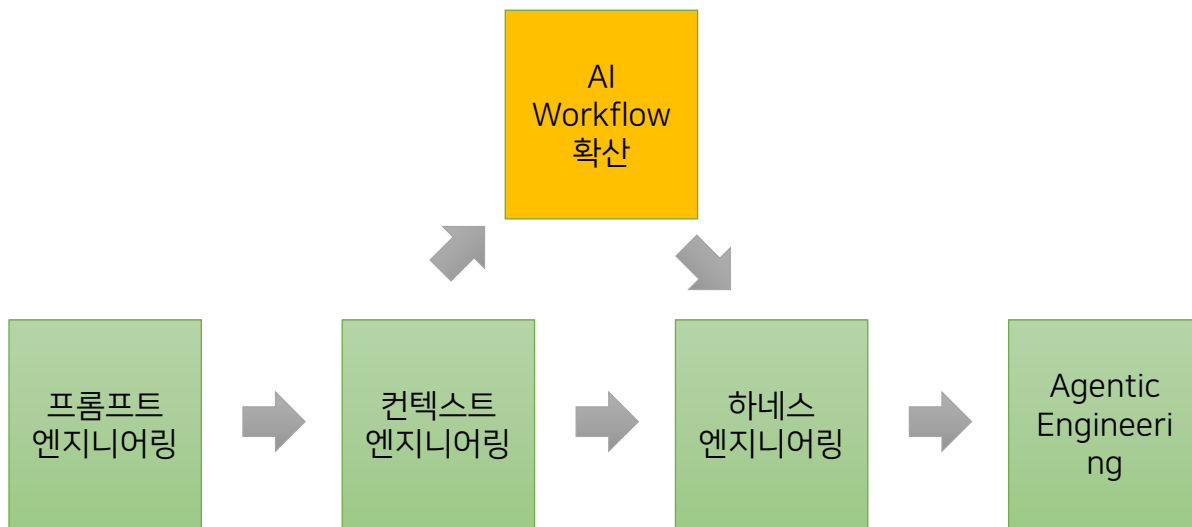
Context Engineering 행동

1. Context Window에 넣은 문서를 구조화 한다.
2. AI가 읽기 쉽게 구역을 나누어 Markdown으로 표시
3. 명확한 지시 내용이 모두 있는지 AI에게 확인시키기

AI Workflow 확산 (2023년)

AI 서비스들은 일회성 대화가 아니라, 사람의 의도를 파악하고, 안전성 검사(가드레일)를 거쳐, 실행 후 결과가 잘 나왔는지 피드백 검증까지 수행한다.

여러 단계의 LLM을 순차적으로 연결해 최종 결과물을 완성하는 프로세스, 이를 'AI Workflow'라고 한다. 이러한 서비스들이 확산되는 시점이었다.



Agent로 구성되는 Workflow 패턴 양상

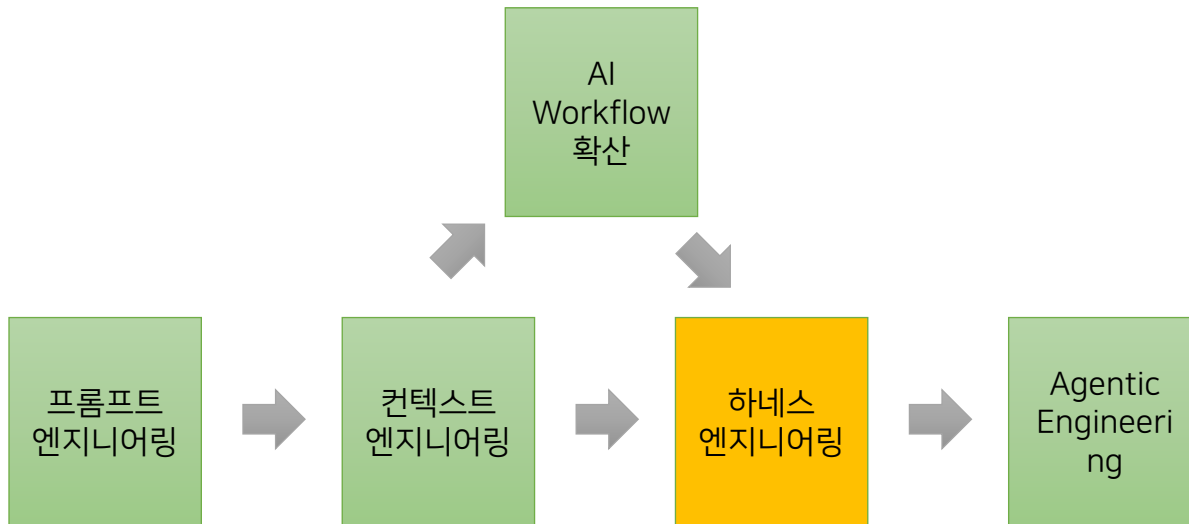
- 순차적(Sequential) Workflow
- 병렬적(Parallel) Workflow
- 평가-개선(Evaluator-optimizer) workflow

Harness Engineering (2025년)

AI Agent에서 “하네스(Harness)”는 Agent가 장시간으로 실행되더라도 올바른 임무를 수행하도록 하는 방법론이다.

에이전트 자체의 성능을 높이는 것보다, Workflow 레벨에서 안전장치를 설계하는 것을 하네스 엔지니어링이라 한다.

여기서 중요한 키워드는 **장시간 실행**이다.



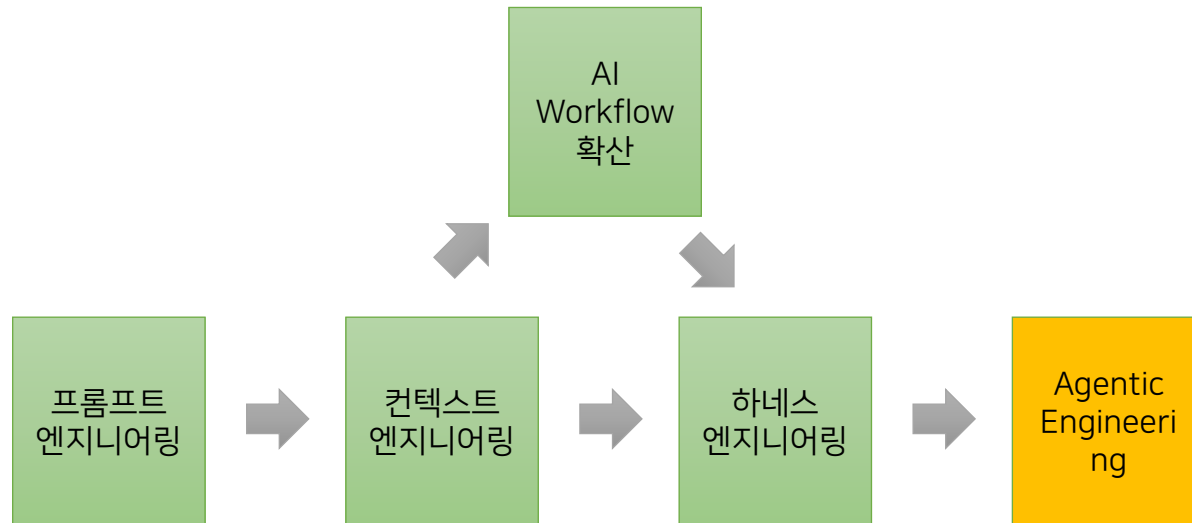
하네스 전 / 후

Agentic Engineering (2026년)

Coding Agent로 인해 생산성은 높아지고, 기능도 잘 동작하지만,

SW 품질 저하로 고객에게 납품하기 힘든 제품이 만들어지는 문제 발생

1. 코드에 대한 이해에 대한 필요성이 적어지기 때문에 의도에 맞지 않는 숨어 있는 버그, 오버엔지니어링이 발생
2. 전체 코드 구조에 대한 이해도가 떨어져, 성능 최적화 포인트나 개선 아이디어를 도출하기 어려워 짐
3. 결과적으로 제품 SW 품질에 대한 신뢰와 자신감이 없음



[도전] Quiz 만점받기

강사님 안내에 따라 Quiz를 풀어주세요.



감사합니다.